



[OTHack1A]

Operational Technology · Automazione & IT

PowerShell

Guida teorico-pratica completa

Dalle origini di MS-DOS allo stato dell'arte

Comandi, scripting, sicurezza e amministrazione Windows Server

con uno sguardo all'integrazione con l'Intelligenza Artificiale

Edizione 2026

Lingua: Italiano · Formato A4

Riferimento: Windows PowerShell 5.1 · PowerShell 7.5 / 7.6

Windows Server 2025

Indice

Teoria e storia	9
La riga di comando: concetti fondamentali	10
Shell, kernel e terminale	10
Perche la riga di comando, nell'epoca delle interfacce grafiche	10
Comando, parametri, REPL	10
Da MS-DOS a Windows: le origini	12
MS-DOS e COMMAND.COM	12
L'arrivo di Windows e di cmd.exe	12
Windows Script Host: un primo tentativo	12
La nascita di PowerShell: il progetto Monad	14
Il Monad Manifesto (2002)	14
Da Monad a PowerShell	14
Perche un nuovo linguaggio e non l'ennesima estensione	14
Evoluzione delle versioni	15
L'era di Windows PowerShell (1.0 - 5.1)	15
L'era moderna: open source e multiplatforma	15
Architettura: perche gli oggetti cambiano tutto	17
La pipeline a oggetti	17
Cmdlet, funzioni, provider	17
Il ruolo di .NET e dell'hosting	17
Stato dell'arte e futuro: PowerShell e l'Intelligenza Artificiale	19
AI Shell	19
Copilot e il Model Context Protocol (MCP)	19
Modelli locali e prospettive	19
Fondamenti pratici	21
Installare e avviare PowerShell	22
Le due edizioni a confronto	22
Installare PowerShell 7	22
Dove scrivere gli script: editor e console	22
Il profilo: personalizzare l'avvio	23

La sintassi di base: la convenzione Verbo-Nome	24
Verbi approvati e nomi	24
Parametri: nominali, posizionali, switch	24
Alias: comodi all'interattivo, da evitare negli script	24
I parametri comuni	25
Il sistema di aiuto: imparare a scoprire i comandi	26
Get-Command: trovare il comando giusto	26
Get-Help: la documentazione integrata	26
Get-Member: cosa contiene un oggetto	26
Gli argomenti about_*	27
La pipeline e gli oggetti in pratica	28
Where-Object: filtrare	28
Select-Object: scegliere proprietà e righe	28
Sort-Object, Group-Object, Measure-Object	28
ForEach-Object: agire su ciascun elemento	28
Formattare l'output	29
Variabili, tipi e operatori	30
Dichiarare e usare variabili	30
I tipi principali	30
Operatori aritmetici e di assegnazione	30
Operatori di confronto	30
Operatori logici e variabili automatiche	31
Provider e PSDrive: un'unica grammatica per archivi diversi	32
I provider integrati	32
Navigare archivi non-file come se fossero cartelle	32
Lavorare con file e cartelle	33
Elencare e filtrare	33
Creare, copiare, spostare, eliminare	33
Leggere e scrivere il contenuto	33
Stringhe, formattazione ed espressioni regolari	35
Virgolette singole e doppie	35
L'operatore di formato -f	35
Metodi delle stringhe e operatori di testo	35
Espressioni regolari: -match e -replace	35

Array, collezioni e tabelle hash	37
Array	37
Tabelle hash	37
Il controllo di flusso	38
Condizioni: if / elseif / else e switch	38
Cicli: for, foreach, while, do	38
Interrompere e proseguire	38
Funzioni, parametri e ambito (scope)	39
Definire una funzione	39
Parametri con il blocco param	39
Funzioni avanzate: CmdletBinding e parametri obbligatori	39
L'ambito delle variabili (scope)	40
La gestione degli errori	41
Errori terminanti e non terminanti	41
try / catch / finally	41
La variabile \$Error e throw	41
Moduli e PowerShell Gallery	43
Trovare, importare e gestire i moduli	43
PowerShell Gallery: installare moduli di terze parti	43
Importare ed esportare dati: CSV, JSON, XML	44
CSV: lo scambio tabellare per eccellenza	44
JSON: configurazioni e API	44
XML e CliXml: fedeltà degli oggetti	44
Sicurezza ed esecuzione degli script	46
La execution policy in dettaglio	47
Che cosa è (e che cosa non è)	47
I tipi di execution policy	47
Leggere e impostare la policy	47
Gli ambiti e la loro precedenza	48
Perché uno script scaricato da Internet non parte	48
Come si aggira (e perché questo prova che non è sicurezza)	48
Mark-of-the-Web: i file che arrivano da fuori	50
Come Windows marchia i file scaricati	50

Sbloccare un file con Unblock-File	50
Firmare digitalmente gli script	51
Il principio	51
Firmare e verificare	51
Credenziali e segreti	52
PSCredential e Get-Credential	52
SecureString: cosa è e cosa non è	52
Il modulo SecretManagement	52
Logging e funzionalità di sicurezza avanzate	54
Registrazione: transcription e script block logging	54
AMSI: l'analisi antimalware in tempo reale	54
Constrained Language Mode e JEA	54
Scripting: dal semplice al complesso	55
Il primo script	56
Struttura di base	56
Eseguire uno script	56
Commenti e aiuto integrato	56
Input, output e parametrizzazione	58
Parametri robusti	58
I canali di output	58
Cicli, file e log in pratica	60
Un log strutturato e riutilizzabile	60
Il ciclo di elaborazione robusto	60
Funzioni avanzate e pipeline	61
Begin, process, end	61
Ricevere proprietà dagli oggetti	61
Le classi in PowerShell	63
Definire una classe	63
Enumerazioni ed ereditarietà	63
Creare un modulo riutilizzabile	65
Il file di modulo .psm1	65
Il manifesto .psd1	65

Qualita, stile e test	67
Buone pratiche di stile	67
PSScriptAnalyzer: il controllo automatico dello stile	67
Pester: scrivere test automatici	67
Attività pianificate e automazione headless	69
Creare un'attività pianificata da PowerShell	69
Gestire e verificare le attività	69
Scrivere script adatti all'esecuzione non presidiata	69
Amministrare Windows Server con PowerShell	71
Amministrare senza interfaccia grafica	72
Server con GUI, Server Core e oltre	72
SConfig e i primi passi su Server Core	72
Windows Admin Center: la GUI moderna basata su comandi	72
PowerShell Remoting: amministrare a distanza	73
Abilitare il Remoting	73
Eseguire comandi remoti: Invoke-Command	73
Sessioni interattive e persistenti	73
Remoting su SSH e multiplatforma	74
Ruoli e funzionalità del server	75
Elencare, installare, rimuovere	75
Installazione coordinata su più server	75
Gestione della rete	76
Indirizzi IP, gateway e DNS	76
Diagnostica di rete	76
Il firewall di Windows	76
Active Directory	78
Consultare utenti, gruppi, computer	78
Creare account e gestire l'appartenenza	78
Creazione di massa da un file CSV	78
Storage, condivisioni e permessi	80
Dischi e volumi	80
Condivisioni SMB	80

Permessi NTFS: leggere e modificare le ACL	80
Servizi, processi ed eventi	81
Servizi	81
Processi	81
Il registro eventi con Get-WinEvent	81
IIS, Hyper-V e configurazione dichiarativa	82
IIS: il web server	82
Hyper-V: le macchine virtuali	82
Desired State Configuration: descrivere lo stato voluto	82
PowerShell nell'automazione industriale	84
Chiamare API REST e servizi web	85
Invoke-RestMethod: leggere e scrivere dati	85
Autenticazione e intestazioni	85
Invoke-WebRequest quando serve la risposta grezza	85
Comunicazione TCP e protocolli industriali	87
Un client TCP grezzo con .NET	87
Esempio: lettura Modbus TCP di registri holding	87
OPC UA e MQTT	87
Porte seriali e strumenti di campo	89
Aprire una porta e leggere	89
Scrivere un comando e attendere la risposta	89
Database e persistenza dei dati	90
SQL Server con il modulo SqlServer	90
Scrivere misure senza dipendenze, con .NET	90
Reportistica: CSV, Excel e PDF	91
CSV: il report universale	91
Excel nativo con il modulo ImportExcel	91
PDF: la via pratica	91
Allarmi e notifiche	93
Notifiche via e-mail	93
Notifiche via webhook (Teams, Telegram, ...)	93
Un monitor completo, in pochi mattoni	93

Confronti tra shell	95
Tre mondi a confronto	96
La differenza di fondo: testo contro oggetti	96
Come leggere le tabelle di equivalenza	96
Equivalenze: file e cartelle	97
Equivalenze: testo, ricerca e dati	98
Equivalenze: processi, servizi e sistema	99
Equivalenze: rete	99
Confronto delle caratteristiche	100
Punti di forza di ciascuna shell	100
Quale scegliere	100
Migrare verso PowerShell	102
Da cmd/batch a PowerShell	102
Da bash a PowerShell	102
Appendici	104
Casi reali di automazione industriale	105
A.1 - Analisi di un log esportato da PLC	105
A.2 - Backup con rotazione e ritenzione	105
A.3 - Monitoraggio di un server SCADA con report e avviso	106
A.4 - Raccolta misure via API e archiviazione storica	106
A.5 - Creazione massiva di operatori in Active Directory	107
Riferimento rapido dei cmdlet	108
Esplorazione e aiuto	108
File, percorsi e contenuti	108
Pipeline e gestione degli oggetti	108
Sistema, servizi e processi	109
Dati, conversioni e rete	109
Sintassi rapida: operatori e variabili	110
Operatori di confronto	110
Operatori logici, aritmetici e di assegnazione	110
Variabili automatiche utili	110
Frammenti utili da ricordare	111

Glossario	112
Fonti e bibliografia	113
Documentazione ufficiale	113
Storia e origini	113
Moduli e strumenti citati	113
Standard e protocolli industriali	113

PARTE I

Teoria e storia

Come nasce e si evolve la shell di Microsoft: dalle origini a riga di comando di MS-DOS fino allo stato dell'arte multiplatforma e all'integrazione con l'Intelligenza Artificiale.

CAPITOLO 1

La riga di comando: concetti fondamentali

Prima di raccontare la storia di PowerShell conviene fissare alcuni concetti, perché torneranno in tutto il testo. Un automatico che programma un PLC sa bene quanto conti distinguere il *campo* (i sensori e gli attuatori) dal *controllore* (la CPU che esegue la logica): allo stesso modo, in informatica, occorre distinguere il **sistema operativo** dal programma che usiamo per impartirgli ordini.

Shell, kernel e terminale

Il **kernel** è il nucleo del sistema operativo: gestisce memoria, processi, file e dispositivi. L'utente non parla mai direttamente con il kernel; lo fa tramite un programma intermedio chiamato **shell** (letteralmente *guscio*, perché avvolge il kernel). La shell legge i comandi, li interpreta, chiede al sistema operativo di eseguirli e mostra i risultati.

Si distingue tra **shell testuale** (CLI, *Command Line Interface*), in cui si digitano comandi, e **shell grafica** (GUI, *Graphical User Interface*), fatta di finestre, icone e mouse. Windows Explorer e la shell grafica di Windows; `cmd.exe` e PowerShell sono shell testuali. Sono due modi di parlare con lo stesso kernel.

Il **terminale** (o **console**) è invece la finestra che ospita la shell: si occupa solo di disegnare i caratteri e raccogliere la tastiera. Su Windows moderno il terminale è *Windows Terminal*; la shell che gira dentro può essere PowerShell, `cmd` o WSL. È una distinzione analoga a quella tra un pannello operatore (HMI) e il programma che gira nel PLC dietro di esso: l'HMI mostra, il PLC elabora.

Perché la riga di comando, nell'epoca delle interfacce grafiche

Per un'attività singola la GUI è comoda: si clicca e si vede subito il risultato. Ma quando l'operazione va **ripetuta** su decine di server, **pianificata** di notte o **documentata** in modo che sia riproducibile identica, la riga di comando diventa insostituibile. Un comando si scrive una volta, si salva in uno script e si riusa mille volte senza errori manuali.

- **Automazione:** un'attività ripetitiva diventa uno script eseguibile a comando o su pianificazione.
- **Riproducibilità:** lo stesso script produce sempre lo stesso risultato, fondamentale negli ambienti industriali e regolamentati.
- **Scalabilità:** la stessa operazione si applica a 1 o 1000 macchine con la medesima fatica.
- **Tracciabilità:** lo script è testo, quindi versionabile, revisionabile e archiviabile come un programma PLC.
- **Accesso remoto:** su un server senza interfaccia grafica (molto comune) la CLI è spesso l'unico modo di operare.

NOTA —

In un impianto di trattamento acque la differenza tra cliccare su una GUI e lanciare uno script è la stessa che passa tra avviare una pompa a mano e affidarla a un automatismo: il secondo approccio è ripetibile, documentato e non dipende dall'operatore di turno.

Comando, parametri, REPL

Una riga di comando ha quasi sempre la stessa struttura: il **nome del comando**, seguito da uno o più **parametri** (detti anche *opzioni* o *switch*) che ne modificano il comportamento, e dagli **argomenti** (i valori su cui agisce). La shell lavora in modalità **REPL** (*Read-Eval-Print Loop*): legge la riga, la valuta, stampa il risultato

e torna in attesa.

Esempio 1.1 - anatomia di alcuni comandi.

```
# Struttura tipica: <comando> -<parametro> <argomento>
Get-Service -Name 'Spooler'          # legge lo stato del servizio di stampa
Copy-Item -Path .\dati.csv -Destination D:\backup\ # copia un file
ping 192.168.1.10 -n 4                # 4 pacchetti verso un dispositivo di campo
```

Questa regolarità non è casuale: come vedremo, in PowerShell è portata all'estremo con la convenzione **Verbo-Nome**, che rende i comandi prevedibili e quasi autoesplicativi.

Fonti

- Microsoft Learn, *What is a command shell?* - learn.microsoft.com/powershell
- Microsoft Learn, *Windows Terminal overview* - learn.microsoft.com/windows/terminal

CAPITOLO 2

Da MS-DOS a Windows: le origini

PowerShell non nasce dal nulla: e la risposta di Microsoft a trent'anni di limiti delle proprie interfacce a riga di comando. Per capirne le scelte progettuali bisogna ripercorrere la linea che parte da MS-DOS.

MS-DOS e COMMAND.COM

All'inizio degli anni '80 **MS-DOS** (*Microsoft Disk Operating System*) era un sistema operativo monoutente e monotask, interamente testuale. La sua shell era **COMMAND.COM**, l'interprete che mostrava il celebre prompt **C:\>** e accettava comandi come **DIR**, **COPY**, **DEL**, **CD**. I comandi erano di due tipi: **interni** (incorporati in **COMMAND.COM**, come **DIR**) ed **esterni** (programmi **.EXE** o **.COM** su disco, come **FORMAT**).

L'automazione si faceva con i **file batch** (**.BAT**): semplici elenchi di comandi eseguiti in sequenza. Il file **AUTOEXEC.BAT** veniva eseguito all'avvio del sistema. Il linguaggio batch era però rudimentale: variabili limitate, nessun vero tipo di dato, gestione degli errori quasi assente, tutto basato su **testo**.

Esempio 2.1 - tipico script batch: poche strutture, solo testo.

```
:: esempio di file batch MS-DOS (sintassi .BAT)
@ECHO OFF
ECHO Backup giornaliero
XCOPY C:\dati D:\backup /S /E /Y
IF ERRORLEVEL 1 ECHO Errore durante la copia
PAUSE
```

L'arrivo di Windows e di cmd.exe

Con Windows NT (1993) e la linea che porta a Windows 2000, XP e successivi, l'interprete dei comandi diventa **cmd.exe**. Rispetto a **COMMAND.COM**, **cmd.exe** è più potente (variabili d'ambiente più ricche, operatori, ridirezioni avanzate), ma resta concettualmente un interprete **orientato al testo**: ogni comando produce flussi di caratteri che il comando successivo deve re-interpretare.

Questo è il limite strutturale: per estrarre un'informazione da un output bisogna *ritagliare* il testo con strumenti come **find** o **findstr**, sperando che il formato non cambi. E come leggere il valore di un sensore non dal registro del PLC, ma interpretando la stringa stampata su un display: fragile e dipendente dalla formattazione.

Windows Script Host: un primo tentativo

Per superare il batch, alla fine degli anni '90 Microsoft introduce **Windows Script Host** (WSH), che permette di scrivere script in **VBScript** o **JScript** con accesso agli oggetti **COM** del sistema. È un salto avanti, ma frammentario: la sintassi è diversa da quella della shell interattiva, l'accesso al sistema passa per oggetti COM eterogenei e la sicurezza è debole (i famigerati virus in VBScript della fine anni '90 nascono proprio da qui).

Resta dunque un vuoto: manca uno strumento **unico** che sia insieme shell interattiva e linguaggio di scripting, potente come l'amministrazione di sistema richiede e coerente nella sintassi. Questo vuoto verrà colmato da PowerShell.

Tabella 2.1 - le shell Microsoft prima di PowerShell.

Caratteristica	MS-DOS (COMMAND.COM)	Windows (cmd.exe)	WSH (VBScript)
Anno	1981	1993 (NT)	1998
Modello dati	Testo	Testo	Oggetti COM
Variabili	Molto limitate	D'ambiente, stringhe	Tipizzate (Variant)

Caratteristica	MS-DOS (COMMAND.COM)	Windows (cmd.exe)	WSH (VBScript)
Gestione errori	ERRORLEVEL	ERRORLEVEL, &&/	On Error Resume Next
Automazione	File .BAT	File .BAT/.CMD	File .VBS/.JS
Coerenza shell/script	Parziale	Parziale	Assente (motore separato)

Fonti

- Microsoft Learn, *cmd* e *Command-line reference* - learn.microsoft.com/windows-server
- Microsoft Learn, *Windows Script Host* - learn.microsoft.com

CAPITOLO 3

La nascita di PowerShell: il progetto Monad

La svolta ha una data e un autore precisi. Nell'agosto del **2002 Jeffrey Snover**, ingegnere Microsoft con un passato nei sistemi Unix, scrive un documento destinato a diventare storico: il **Monad Manifesto**.

Il Monad Manifesto (2002)

Nel manifesto Snover parte da un'osservazione semplice: il modello delle shell Unix - piccoli strumenti che si concatenano con la **pipeline** - è geniale, ma è prigioniero del **testo**. Ogni programma stampa testo e il successivo deve ri-analizzarlo con **grep**, **awk**, **sed**. Snover propone di mantenere l'idea della pipeline ma di far scorrere lungo di essa non testo, bensì **oggetti strutturati**.

L'intuizione è potente: se un comando restituisce oggetti dotati di **proprietà** (i dati) e **metodi** (le azioni), il comando successivo non deve indovinare dove finisce una colonna e ne inizia un'altra: accede direttamente alla proprietà che gli serve. Il sistema operativo Windows era ormai costruito su **.NET**, che offriva proprio un mondo di oggetti tipizzati: Monad li avrebbe esposti all'amministratore.

SUGGERIMENTO —

L'idea chiave - *oggetti invece di testo nella pipeline* - è ciò che ancora oggi distingue PowerShell da qualsiasi shell tradizionale. È il «registro tipizzato» invece della «stringa sul display».

Da Monad a PowerShell

Il progetto, nome in codice **Monad**, viene presentato pubblicamente per la prima volta alla *Professional Developers Conference* di Los Angeles nell'ottobre **2003**. Il comando si chiamava allora **msh** (*Microsoft Shell*). Dopo anni di sviluppo e beta, nell'aprile **2006** Microsoft annuncia il nome definitivo - **Windows PowerShell** - con un celebre post intitolato, con un gioco di parole, *Windows PowerShell (Monad) Has Arrived*. La presentazione ufficiale avviene al TechEd Europe di Barcellona.

La versione **1.0** di Windows PowerShell viene rilasciata il **14 novembre 2006**, con **129 cmdlet** di base e la pipeline a oggetti come cuore del sistema. Il successo è rapido: circa un milione di download nei primi sei mesi. Snover sarebbe rimasto per anni l'architetto di riferimento della tecnologia, lasciando poi Microsoft nel 2022.

Perché un nuovo linguaggio e non l'ennesima estensione

Microsoft avrebbe potuto limitarsi a potenziare **cmd.exe**. La scelta di ricominciare da capo rispondeva a obiettivi di lungo periodo: avere un motore costruito su **.NET**, una sintassi coerente tra uso interattivo e script, e soprattutto un modello a oggetti che rendesse l'amministrazione **programmabile** in modo solido. La stessa logica per cui, in automazione, a un certo punto si abbandona la logica a rele cablata per passare al PLC: non è un miglioramento incrementale, è un cambio di paradigma.

Fonti

- Snover J., *Monad Manifesto*, agosto 2002 - ripubblicato su devblogs.microsoft.com
- Microsoft, *Windows PowerShell (Monad) Has Arrived*, 25 aprile 2006 - devblogs.microsoft.com
- Microsoft Learn, *A Brief History of PowerShell* - learn.microsoft.com/powershell

CAPITOLO 4

Evoluzione delle versioni

Dal 2006 a oggi PowerShell ha attraversato due grandi ere: quella di **Windows PowerShell** (versioni 1.0-5.1, solo Windows, basate sul .NET Framework) e quella di **PowerShell** moderno (dalla 6.0 in poi, multiplatforma e open source, basato su .NET Core / .NET).

L'era di Windows PowerShell (1.0 - 5.1)

- **1.0 (2006)** - pipeline a oggetti, 129 cmdlet, integrato in Windows Server 2008.
- **2.0 (2009)** - introduce il **Remoting** (WinRM), i moduli, i job in background, l'ISE (l'editor integrato). Distribuito con Windows 7 e Server 2008 R2.
- **3.0 (2012)** - sintassi semplificata, *workflow*, miglior scoperta dei comandi. Windows 8 / Server 2012.
- **4.0 (2013)** - **Desired State Configuration** (DSC), per descrivere lo stato voluto di un sistema in modo dichiarativo.
- **5.0 / 5.1 (2016)** - classi PowerShell, gestione pacchetti (PackageManagement, PowerShellGet). La **5.1** è l'ultima versione di *Windows PowerShell* ed è tuttora preinstallata in Windows 10/11 e Windows Server.

ATTENZIONE —

Windows PowerShell **5.1** è l'ultima della sua linea: non riceverà nuove funzionalità, solo correzioni di sicurezza. E però ancora presente di default su ogni Windows, quindi va conosciuta - tipicamente convive con PowerShell 7 sulla stessa macchina.

L'era moderna: open source e multiplatforma

Il **18 agosto 2016** Microsoft compie il passo che pochi avrebbero previsto: rende PowerShell **open source** e lo porta su **Linux** e **macOS**, ribattezzandolo inizialmente *PowerShell Core*. Il motore viene riscritto su .NET Core, multiplatforma.

- **6.0 (2018)** - prima versione cross-platform (Windows, Linux, macOS), su .NET Core. Cade il suffisso storico: il prodotto è semplicemente *PowerShell*.
- **7.0 (2020)** - su .NET Core 3.1; obiettivo dichiarato: sostituire Windows PowerShell 5.1 per i nuovi sviluppi. Reintrodotta compatibilità con molti moduli di 5.1.
- **7.2 - 7.4 (2021-2023)** - versioni LTS su .NET 6 e .NET 8, prestazioni e stabilità.
- **7.5 (febbraio 2025)** - basata su **.NET 9**; tra le novità i cmdlet `ConvertTo-CliXml` e `ConvertFrom-CliXml`.

A metà del 2026 la linea moderna prosegue con versioni **7.5** (stabile) e **7.6** (LTS) affiancate; sono in arrivo le anteprime della 7.7. La numerazione è ormai disaccoppiata da quella di Windows: PowerShell è un prodotto a se, installabile e aggiornabile in modo indipendente.

Tabella 4.1 - le tappe principali delle versioni di PowerShell.

Versione	Anno	Base .NET	Piattaforme	Novità chiave
1.0	2006	.NET Framework	Windows	Pipeline a oggetti, 129 cmdlet
2.0	2009	.NET Framework	Windows	Remoting, moduli, ISE
3.0	2012	.NET Framework	Windows	Workflow, sintassi semplice
4.0	2013	.NET Framework	Windows	DSC

Versione	Anno	Base .NET	Piattaforme	Novita chiave
5.1	2016	.NET Framework	Windows	Classi, PackageManagement (ultima 'Windows PowerShell')
6.0	2018	.NET Core	Win/Linux/macOS	Open source, multipiattaforma
7.0	2020	.NET Core 3.1	Win/Linux/macOS	Sostituto di 5.1, compatibilita moduli
7.4	2023	.NET 8 (LTS)	Win/Linux/macOS	Stabilita, prestazioni
7.5	2025	.NET 9	Win/Linux/macOS	ConvertTo/From-CliXml

NOTA —

In pratica, su una postazione di lavoro o un server moderni si trovano spesso **due** PowerShell: la 5.1 di sistema (comando `powershell.exe`) e la 7.x installata a parte (comando `pwsh.exe`). Sono affiancate, non in conflitto.

Fonti

- Microsoft, *PowerShell is open sourced and is available on Linux*, 18 agosto 2016 - azure.microsoft.com/blog
- Microsoft Learn, *What's New in PowerShell 7.5* - learn.microsoft.com/powershell
- Microsoft Learn, *PowerShell support lifecycle* - learn.microsoft.com/powershell

CAPITOLO 5

Architettura: perche gli oggetti cambiano tutto

Capire l'architettura di PowerShell e cio che separa chi *copia* comandi da chi li *padroneggia*. Il cuore e uno: nella pipeline non scorre testo, scorrono **oggetti .NET**.

La pipeline a oggetti

Una pipeline collega piu comandi con il simbolo `|`: l'output di uno diventa l'input del successivo. In una shell tradizionale quell'output e testo; in PowerShell e una sequenza di oggetti completi di proprieta e metodi. Il comando a valle non deve *ritagliare* nulla: interroga direttamente le proprieta.

Esempio 5.1 - la pipeline manipola oggetti per proprieta (.Name, .Id, .WorkingSet).

```
# Ferma tutti i processi 'notepad' attivi, ordinati per memoria occupata.  
# Lungo la pipeline scorrono oggetti Process, non righe di testo:  
Get-Process -Name notepad |  
    Sort-Object -Property WorkingSet -Descending |  
    Select-Object -First 3 Name, Id, WorkingSet |  
    Stop-Process -WhatIf
```

In `cmd` o in `bash` la stessa operazione richiederebbe di stampare la lista, filtrarla con strumenti testuali e sperare che le colonne non cambino posizione. Qui invece `WorkingSet` e una **proprieta numerica** dell'oggetto: l'ordinamento e esatto, non lessicografico.

Cmdlet, funzioni, provider

I comandi nativi di PowerShell si chiamano **cmdlet** (si pronuncia *command-let*). Non sono eseguibili separati come in DOS: sono classi .NET che il motore carica e che seguono tutte la convenzione **Verbo-Nome** (`Get-Process`, `Set-Item`, `New-Service`). Questo rende il sistema **prevedibile**: intuito un verbo, si indovinano molti comandi.

- **Cmdlet** - comandi compilati (.NET) che seguono Verbo-Nome.
- **Funzioni** - comandi scritti in PowerShell stesso, definiti dall'utente o dai moduli.
- **Provider** - adattatori che espongono archivi di dati (file system, registro di Windows, certificati, variabili) come fossero unita disco navigabili con `cd` e `dir`.
- **Moduli** - pacchetti che raggruppano cmdlet, funzioni e provider su un tema (es. il modulo di Active Directory).

L'idea di **provider** e particolarmente elegante: grazie ad essa si puo «navigare» il registro di Windows come una cartella (`cd HKLM:\SOFTWARE`) usando gli stessi comandi che si usano per i file. Un'unica grammatica per archivi diversi.

Il ruolo di .NET e dell'hosting

PowerShell e costruito sopra la piattaforma **.NET**: ogni dato e in ultima analisi un oggetto .NET, e dalla shell si possono richiamare direttamente classi e metodi del framework (ad esempio `[math]::Sqrt(2)` o `[datetime]::Now`). Il motore puo inoltre essere **ospitato** (hosting) dentro altre applicazioni: e cosi che molti prodotti server offrono una loro console basata su PowerShell.

Esempio 5.2 - PowerShell e una porta d'accesso interattiva a .NET.

```
# Accesso diretto alle classi .NET dalla shell:
[math]::Round([math]::PI, 4)      # -> 3.1416
[datetime]::Now.AddDays(30)      # data tra 30 giorni
[System.Environment]::OSVersion  # versione del sistema operativo
```

Fonti

- Microsoft Learn, *Understanding the PowerShell pipeline* - learn.microsoft.com/powershell
- Microsoft Learn, *about_Providers* - learn.microsoft.com/powershell

CAPITOLO 6

Stato dell'arte e futuro: PowerShell e l'Intelligenza Artificiale

Oggi PowerShell è maturo, multiplatforma e open source. La frontiera attuale è l'integrazione con **l'Intelligenza Artificiale**: assistenti che aiutano a scrivere comandi, a interpretare gli errori e a orchestrare strumenti esterni.

AI Shell

Microsoft ha introdotto **AI Shell**, un progetto che porta un assistente conversazionale accanto alla riga di comando. Si compone di un eseguibile autonomo (**aish**) e di un modulo PowerShell che lo affianca alla sessione come una barra laterale (ad esempio dentro Windows Terminal). L'idea è poter chiedere in linguaggio naturale come si fa una certa operazione e ricevere il comando pronto, senza lasciare il terminale.

- Architettura ad **agenti**: ad esempio un agente basato su modelli *Azure OpenAI* e uno legato a *Copilot in Azure*.
- Un cmdlet come **Resolve-Error** consente di passare all'assistente l'ultimo errore perché lo spieghi e proponga una correzione.
- Funziona affiancato alla shell, mantenendo il contesto dei comandi recenti.

ATTENZIONE —

AI Shell è in fase di **anteprima** (preview): utile e promettente, ma non ancora uno strumento stabile da produzione. Va valutato in ambiente di test, mai dato per definitivo.

Copilot e il Model Context Protocol (MCP)

Sul fronte assistenti, **GitHub Copilot** e i *Copilot* di Microsoft sanno generare e spiegare script PowerShell direttamente nell'editor. Più interessante a livello architetturale è l'adozione crescente del **Model Context Protocol (MCP)**, un protocollo aperto nato per standardizzare il modo in cui un assistente IA si collega a strumenti e fonti di dati esterni. Sia AI Shell sia, in prospettiva, Windows stanno integrando il supporto a server MCP, aprendo la strada ad assistenti capaci di agire su strumenti reali in modo controllato.

Per un tecnico dell'automazione l'analogia è immediata: MCP è una sorta di **bus di campo per gli assistenti IA**: come Modbus o OPC UA standardizzano il dialogo tra dispositivi di fornitori diversi, MCP standardizza il dialogo tra un modello e gli strumenti che deve comandare.

Modelli locali e prospettive

Accanto ai servizi in cloud cresce l'interesse per i **modelli locali**, che girano sulla macchina (anche sfruttando NPU e GPU) senza inviare dati all'esterno: aspetto cruciale dove la riservatezza o l'isolamento di rete sono requisiti, come nelle reti OT industriali. Microsoft sta muovendosi in questa direzione con iniziative dedicate ai modelli eseguiti localmente su Windows.

La prospettiva non è un'IA che *sostituisce* l'amministratore, ma un **copilota**: suggerisce il comando, spiega l'errore, redige la prima bozza di uno script. La responsabilità di leggere, capire ed eseguire resta - e deve restare - di chi sta alla tastiera. Esattamente come un sistema esperto può proporre una taratura, ma è il tecnico a validarla prima di mandarla in campo.

SUGGERIMENTO —

Regola pratica con qualsiasi assistente IA: non eseguire mai un comando che non comprendi. L'IA accelera chi già sa leggere il codice; non è una scorciatoia per saltarne la comprensione.

Fonti

- Microsoft Learn / GitHub, *AI Shell* (progetto in preview) - learn.microsoft.com, github.com/PowerShell/AIShell
- Anthropic, *Model Context Protocol* (specifica aperta) - modelcontextprotocol.io
- Microsoft, documentazione su *Windows AI Foundry* e modelli locali - learn.microsoft.com/windows/ai

PARTE II

Fondamenti pratici

Installazione, sintassi, sistema di aiuto, pipeline, variabili e tipi: le basi solide su cui costruire qualunque script. Tutti i comandi spiegati con esempi eseguibili.

CAPITOLO 7

Installare e avviare PowerShell

Su Windows PowerShell è già presente: la versione **5.1** è preinstallata e non richiede nulla. La versione **7.x** moderna si installa a parte e convive con la 5.1. È importante sapere quale si sta usando, perché alcuni moduli e comportamenti differiscono.

Le due edizioni a confronto

Si riconoscono dal nome dell'eseguibile e da una variabile automatica:

Esempio 7.1 - identificare versione ed edizione.

```
# Quale versione sto usando?
$PSVersionTable.PSVersion      # versione completa
$PSVersionTable.PSEdition      # 'Desktop' = 5.1 ; 'Core' = 7.x

# Gli eseguibili sono diversi:
# powershell.exe -> Windows PowerShell 5.1 (Desktop)
# pwsh.exe       -> PowerShell 7.x (Core, multiplatforma)
```

Tabella 7.1 - quando usare l'una o l'altra. Per nuovi progetti: 7.x.

Aspetto	Windows PowerShell 5.1	PowerShell 7.x
Eseguibile	powershell.exe	pwsh.exe
Base	.NET Framework	.NET (Core)
Piattaforme	Solo Windows	Windows, Linux, macOS
Installazione	Preinstallata	Da installare a parte
Sviluppo	Solo correzioni	Attivo, nuove funzionalità
Edizione (\$PSEdition)	Desktop	Core

Installare PowerShell 7

Su Windows il modo più pulito è tramite **winget**, il gestore pacchetti di Windows. In alternativa si scarica il pacchetto MSI dal repository ufficiale su GitHub.

Esempio 7.2 - installare PowerShell 7 con winget.

```
# Installazione/aggiornamento via winget (consigliato su Windows):
winget install --id Microsoft.PowerShell --source winget

# Verifica dopo l'installazione:
pwsh -Version
```

NOTA —

A partire dalle anteprime più recenti, Microsoft sta deprecando la distribuzione tramite pacchetti MSI a favore di altri canali (winget, Store, archivi). Verificare sempre la pagina ufficiale del progetto per il metodo corrente.

Dove scrivere gli script: editor e console

- **Console interattiva** - per comandi rapidi e prove. E il REPL: scrivi, premi Invio, vedi il risultato.

- **Windows Terminal** - terminale moderno che ospita piu shell (PowerShell, cmd, WSL) in schede; consigliato come terminale predefinito.
- **Visual Studio Code** con l'estensione *PowerShell* - l'ambiente di sviluppo raccomandato oggi: evidenziazione, completamento, debug passo-passo.
- **Windows PowerShell ISE** - il vecchio editor integrato; funziona solo con la 5.1 e non riceve piu sviluppo. Da conoscere ma non da preferire per il nuovo.

SUGGERIMENTO —

Per chi viene dall'automazione: VS Code sta a PowerShell come un ambiente di sviluppo PLC sta al codice di automazione. La console e ottima per il test rapido di un comando, ma il progetto vero si scrive e si versiona nell'editor.

Il profilo: personalizzare l'avvio

Ogni sessione puo eseguire automaticamente uno script di avvio detto **profilo** - l'equivalente dell'**AUTOEXEC.BAT** di un tempo, ma molto piu potente. Vi si mettono alias personali, funzioni di uso frequente, impostazioni dell'aspetto.

Esempio 7.3 - individuare e creare il profilo.

```
# Percorso del profilo dell'utente corrente:
$PROFILE

# Crearlo se non esiste e aprirlo nell'editor:
if (-not (Test-Path $PROFILE)) { New-Item -ItemType File -Path $PROFILE -Force }
code $PROFILE      # oppure notepad $PROFILE
```

Fonti

- Microsoft Learn, *Installing PowerShell on Windows* - learn.microsoft.com/powershell
- Microsoft Learn, *about_Profiles* - learn.microsoft.com/powershell

CAPITOLO 8

La sintassi di base: la convenzione Verbo-Nome

La forza didattica di PowerShell sta nella sua **regolarità**. Quasi ogni comando ha la forma **Verbo-Nome**: il verbo dice cosa si fa, il nome dice *su cosa*. Imparata la grammatica, gran parte del vocabolario si deduce.

Verbi approvati e nomi

Microsoft mantiene un elenco di **verbi approvati**, raggruppati per significato (Get, Set, New, Remove, Start, Stop, ...). Usarli con coerenza rende gli script leggibili a chiunque conosca lo standard.

Tabella 8.1 - alcuni verbi approvati e il loro uso. `Get-Verb` li elenca tutti.

Verbo	Significato	Esempi di cmdlet
Get	Leggere/ottenere	Get-Process, Get-Service, Get-Content
Set	Modificare/impostare	Set-Location, Set-Content, Set-Service
New	Creare	New-Item, New-Service, New-Object
Remove	Eliminare	Remove-Item, Remove-Service
Start / Stop	Avviare / fermare	Start-Service, Stop-Process
Add / Clear	Aggiungere / svuotare	Add-Content, Clear-Content
Import / Export	Importare / esportare	Import-Csv, Export-Csv
Test	Verificare una condizione	Test-Path, Test-Connection

Esempio 8.1 - scoprire i verbi standard.

```
# L'elenco completo dei verbi approvati e raggruppati per categoria:
Get-Verb | Sort-Object Verb | Format-Table Verb, Group -AutoSize
```

Parametri: nominali, posizionali, switch

Gli argomenti si passano tramite **parametri**. Un parametro **nominale** si indica con **-Nome valore**; uno **posizionale** si passa nell'ordine previsto, senza nome; uno **switch** è un interruttore senza valore (presente = vero).

Esempio 8.2 - le tre forme di parametro.

```
# Nominale (esplicito, sempre consigliato negli script):
Get-ChildItem -Path 'C:\Logs' -Filter '*.log' -Recurse

# Posizionale (Path è il primo parametro posizionale di Get-ChildItem):
Get-ChildItem 'C:\Logs'

# Switch (interruttore: la sua presenza attiva il comportamento):
Get-ChildItem -Path 'C:\Logs' -Recurse -Force
```

ATTENZIONE —

Negli script usare **sempre** i parametri nominali, anche quando quelli posizionali funzionano. Uno script che si appoggia all'ordine posizionale è fragile e di difficile lettura, esattamente come del codice PLC con indirizzi assoluti al posto di simboli.

Alias: comodi all'interattivo, da evitare negli script

Molti cmdlet hanno **alias** brevi, spesso pensati per chi viene da DOS o Unix: `dir`, `ls`, `gci` sono tutti alias di `Get-ChildItem`; `cd` è alias di `Set-Location`. Ottimi per digitare in fretta, da evitare negli script per chiarezza.

Esempio 8.3 - alias e loro risoluzione.

```
# Questi tre comandi sono identici:
Get-ChildItem
dir
ls

# Scoprire a cosa corrisponde un alias e quali alias ha un cmdlet:
Get-Alias dir
Get-Alias -Definition Get-ChildItem
```

I parametri comuni

Ogni cmdlet ben scritto eredita automaticamente i **parametri comuni**, che danno un comportamento uniforme. I più importanti per lavorare in sicurezza sono `-WhatIf` e `-Confirm`.

- `-WhatIf` - simula l'operazione e mostra cosa *farebbe*, senza eseguirla. Indispensabile prima di comandi distruttivi.
- `-Confirm` - chiede conferma prima di agire.
- `-Verbose` - stampa messaggi dettagliati sull'esecuzione.
- `-ErrorAction` - decide come reagire agli errori (Continue, Stop, SilentlyContinue, ...).
- `-OutVariable` - salva l'output in una variabile oltre a restituirlo.

Esempio 8.4 - `-WhatIf` come rete di sicurezza.

```
# Prima si SIMULA con -WhatIf, poi (solo se l'esito è corretto) si esegue:
Get-ChildItem 'D:\temp' -Filter '*.tmp' | Remove-Item -WhatIf
# -> Operazione "Rimozione file" sull'elemento ... (nessuna modifica reale)
```

SUGGERIMENTO —

La coppia `-WhatIf` / `-Confirm` è la cintura di sicurezza dell'amministratore: prendere l'abitudine di provare ogni comando distruttivo con `-WhatIf` evita disastri, come il *dry-run* di una sequenza prima di mandarla in automatico sull'impianto.

Fonti

- Microsoft Learn, *Approved Verbs for PowerShell Commands* - learn.microsoft.com/powershell
- Microsoft Learn, *about_CommonParameters* - learn.microsoft.com/powershell

CAPITOLO 9

Il sistema di aiuto: imparare a scoprire i comandi

Nessuno ricorda migliaia di cmdlet. La vera competenza non è memorizzarli, ma saperli **scoprire**. PowerShell offre tre comandi che, insieme, permettono di esplorare qualsiasi cosa: **Get-Command**, **Get-Help**, **Get-Member**. Sono la chiave per mantenere la promessa di questa guida - conoscere *tutti* i comandi - senza impararli a memoria.

Get-Command: trovare il comando giusto

Get-Command elenca i comandi disponibili. Si può filtrare per verbo, per nome (con caratteri jolly) o per modulo.

Esempio 9.1 - Get-Command, la bussola dei comandi.

```
# Tutti i comandi che 'leggono' qualcosa (verbo Get):
Get-Command -Verb Get

# Tutti i comandi che riguardano i servizi:
Get-Command -Noun Service

# Cerca per nome con jolly (qualsiasi cosa contenga 'network'):
Get-Command *network*

# Da quale modulo proviene un comando?
(Get-Command Get-Process).ModuleName
```

Get-Help: la documentazione integrata

Get-Help mostra la guida di un comando: sintassi, descrizione, parametri ed **esempi**. La prima volta conviene aggiornare i file di aiuto con **Update-Help**.

Esempio 9.2 - i livelli di Get-Help.

```
# Aggiornare i file di guida (richiede privilegi e connessione):
Update-Help -Force -ErrorAction SilentlyContinue

# Diverse profondità di aiuto:
Get-Help Get-Service           # sintesi
Get-Help Get-Service -Examples # solo gli esempi pratici
Get-Help Get-Service -Detailed # descrizioni dei parametri
Get-Help Get-Service -Full     # tutto
Get-Help Get-Service -Online   # apre la pagina web ufficiale
```

SUGGERIMENTO —

Get-Help <comando> -Examples è probabilmente il comando più utile per imparare: mostra subito casi d'uso reali. Da usare ogni volta che si incontra un cmdlet nuovo.

Get-Member: cosa contiene un oggetto

Poiché nella pipeline scorrono oggetti, la domanda chiave è spesso *quali proprietà e metodi ha questo oggetto?* La risposta la dà **Get-Member**: si manda l'oggetto nella pipeline e si scopre la sua struttura.

Esempio 9.3 - *Get-Member* rivela la struttura degli oggetti.

```
# Quali proprietà/metodi ha un oggetto Service?
Get-Service | Get-Member

# Solo le proprietà:
Get-Service | Get-Member -MemberType Property

# Conoscendo i nomi, si selezionano le proprietà che interessano:
Get-Service | Select-Object Name, Status, StartType
```

Questi tre comandi formano un metodo di lavoro: **Get-Command** per **trovare** il comando, **Get-Help** per **capire** come si usa, **Get-Member** per **esplorare** cosa restituisce. Con essi si naviga qualsiasi modulo, anche sconosciuto.

Gli argomenti `about_*`

Oltre alla guida dei singoli comandi esistono gli articoli **concettuali** `about_*`, che spiegano interi argomenti (operatori, flusso, funzioni, ...). Sono una piccola enciclopedia inclusa nel prodotto.

Esempio 9.4 - *la documentazione concettuale `about_*`.*

```
# Elenco di tutti gli argomenti concettuali:
Get-Help about_*

# Leggere un argomento specifico:
Get-Help about_Operators
Get-Help about_Functions_Advanced
```

Fonti

- Microsoft Learn, *Get-Help, Get-Command, Get-Member* - learn.microsoft.com/powershell
- Microsoft Learn, *about_Updatable_Help* - learn.microsoft.com/powershell

CAPITOLO 10

La pipeline e gli oggetti in pratica

Abbiamo visto il principio della pipeline a oggetti; ora i comandi che la rendono utile ogni giorno: filtrare, selezionare, ordinare, raggruppare, misurare, iterare. Sono i «mattoni» di quasi ogni one-liner.

Where-Object: filtrare

Where-Object (alias `?`) lascia passare solo gli oggetti che soddisfano una condizione. La variabile speciale `$_` (o `$PSItem`) rappresenta l'oggetto corrente.

Esempio 10.1 - Where-Object e la variabile \$_.

```
# Solo i servizi attualmente in esecuzione:
Get-Service | Where-Object { $_.Status -eq 'Running' }

# Sintassi semplificata (dalla 3.0 in poi) per confronti semplici:
Get-Service | Where-Object Status -eq 'Running'

# Processi che usano piu di 200 MB di memoria:
Get-Process | Where-Object { $_.WorkingSet -gt 200MB }
```

Select-Object: scegliere proprieta e righe

Select-Object (alias `select`) sceglie **quali proprieta** mostrare, limita il numero di oggetti (`-First`, `-Last`) e crea proprieta calcolate con una *hashtable*.

Esempio 10.2 - Select-Object con proprieta calcolate.

```
# Solo alcune colonne, primi 5 risultati:
Get-Process | Select-Object -First 5 Name, Id, CPU

# Proprieta calcolata: memoria in MB con etichetta personalizzata
Get-Process | Select-Object Name,
    @{ Name = 'MemoriaMB'; Expression = { [math]::Round($_.WorkingSet/1MB, 1) } } |
    Sort-Object MemoriaMB -Descending | Select-Object -First 10
```

Sort-Object, Group-Object, Measure-Object

Tre comandi per riordinare, raggruppare e fare statistiche sugli oggetti.

Esempio 10.3 - ordinamento, raggruppamento, misura.

```
# Ordinare (numericamente, perche sono proprieta tipizzate):
Get-Process | Sort-Object CPU -Descending | Select-Object -First 5

# Raggruppare i servizi per stato e contarli:
Get-Service | Group-Object Status | Select-Object Name, Count

# Statistiche su una proprieta numerica:
Get-Process | Measure-Object WorkingSet -Sum -Average -Maximum
```

ForEach-Object: agire su ciascun elemento

ForEach-Object (alias `%`) esegue un blocco di codice per ogni oggetto della pipeline. E il modo «scorrevole» di iterare, diverso dal costrutto `foreach` che vedremo piu avanti.

Esempio 10.4 - *ForEach-Object*, anche con *begin/process/end*.

```
# Per ogni file di log, stampa nome e dimensione in KB:
Get-ChildItem 'C:\Logs' -Filter '*.log' | ForEach-Object {
    '{0} -> {1:N1} KB' -f $_.Name, ($_.Length / 1KB)
}

# Forma con blocchi begin/process/end (utile per inizializzare contatori):
1..5 | ForEach-Object -Begin { $tot = 0 } -Process { $tot += $_ } -End { $tot }
```

NOTA —

Differenza pratica tra **ForEach-Object** (cmdlet della pipeline) e **foreach** (parola chiave del linguaggio): il primo elabora gli oggetti **mentre scorrono** ed è ideale in pipeline lunghe; il secondo lavora su una collezione già in memoria. Su grandi volumi la differenza ha effetti su memoria e prestazioni.

Formattare l'output

Alla fine della pipeline si decide come **mostrare** i dati: tabella, lista, o esportazione. Importante: i comandi **Format-*** servono solo alla visualizzazione e vanno **sempre per ultimi**, perché distruggono la struttura a oggetti.

Esempio 10.5 - *Format-Table*, *Format-List*, *Out-GridView*.

```
Get-Service | Format-Table Name, Status -AutoSize    # tabella compatta
Get-Service Spooler | Format-List *                 # tutti i campi, a lista
Get-Process | Out-GridView                          # finestra interattiva (solo Windows)
```

ATTENZIONE —

Non mettere mai un **Format-*** in mezzo alla pipeline seguito da altri comandi che si aspettano oggetti: dopo la formattazione non ci sono più oggetti veri, solo istruzioni di disegno. Prima si elabora, poi - per ultimo - si formatta.

Fonti

- Microsoft Learn, *Where-Object*, *Select-Object*, *ForEach-Object* - learn.microsoft.com/powershell
- Microsoft Learn, *about_Pipelines* - learn.microsoft.com/powershell

CAPITOLO 11

Variabili, tipi e operatori

Le variabili conservano valori; in PowerShell ogni valore è un **oggetto .NET** con un tipo preciso. Comprendere i tipi evita errori sottili, come confrontare un numero con una stringa.

Dichiarare e usare variabili

Una variabile inizia con **\$**. Non serve dichiararne il tipo: PowerShell lo deduce. Si può però **vincolarlo** antepoendo il tipo tra parentesi quadre, cosa consigliata quando il tipo conta.

Esempio 11.1 - variabili, inferenza e vincolo di tipo.

```
$nome = 'Impianto Nord'      # stringa
$portata = 42.5              # numero a virgola mobile (double)
$attivo = $true              # booleano

# Variabile a tipo vincolato: accetterà solo interi
[int]$contatore = 0
$contatore = '10'            # la stringa '10' viene convertita in intero 10

# Conoscere il tipo di un valore:
$portata.GetType().FullName  # System.Double
```

I tipi principali

Tabella 11.1 - tipi di dato di uso comune.

Tipo	Sigla	Esempio	Uso tipico
Stringa	[string]	'testo'	Testi, nomi, percorsi
Intero	[int]	42	Conteggi, indici
Decimale	[double]	3.14	Misure, calcoli
Booleano	[bool]	\$true / \$false	Condizioni, flag
Data/ora	[datetime]	Get-Date	Timestamp, scadenze
Array	[array]	1,2,3	Elenchi ordinati
Tabella hash	[hashtable]	@{a=1}	Coppie chiave-valore

Operatori aritmetici e di assegnazione

Esempio 11.2 - aritmetica e moltiplicatori di byte integrati.

```
5 + 3      # 8      somma
10 % 3     # 1      resto (modulo)
2 * 1KB    # 2048   PowerShell conosce KB, MB, GB, TB, PB
$x = 10; $x += 5    # $x ora vale 15 (anche -= *= /=)
```

Operatori di confronto

Qui PowerShell sorprende chi viene da altri linguaggi: i confronti **non** usano **>** o **==**, ma sigle testuali. Il motivo storico è che **>** significa già *ridirezione su file*.

Tabella 11.2 - operatori di confronto. Niente '>' o '==': si usano le sigle.

Operatore	Significato	Esempio (risultato)
-eq / -ne	uguale / diverso	5 -eq 5 (True)
-gt / -ge	maggiore / maggiore o uguale	7 -gt 3 (True)
-lt / -le	minore / minore o uguale	2 -lt 9 (True)
-like	confronto con jolly	'log.txt' -like '*.txt' (True)
-match	espressione regolare	'abc123' -match '\d+' (True)
-contains / -in	appartenenza a collezione	1,2,3 -contains 2 (True)

ATTENZIONE —

Errore classico: scrivere `if ($a > 5)`. In PowerShell > **ridirige** l'output su un file di nome 5! La forma corretta è `if ($a -gt 5)`. È una delle prime cose che confonde chi arriva da C, Java o dal mondo PLC strutturato.

Operatori logici e variabili automatiche

Gli operatori logici sono `-and`, `-or`, `-not` (o `!`). Esistono poi molte **variabili automatiche** già pronte, utili da conoscere.

Esempio 11.3 - operatori logici e variabili automatiche.

```
if ($attivo -and ($portata -gt 40)) { 'Impianto in regime' }

# Alcune variabili automatiche utili:
$PSVersionTable # versione di PowerShell
$PWD            # cartella corrente
$HOME          # cartella personale dell'utente
$?             # esito dell'ultimo comando (True/False)
$PID           # ID del processo PowerShell corrente
```

Fonti

- Microsoft Learn, *about_Variables*, *about_Operators*, *about_Comparison_Operators* - learn.microsoft.com/powershell
- Microsoft Learn, *about_Automatic_Variables* - learn.microsoft.com/powershell

CAPITOLO 12

Provider e PSDrive: un'unica grammatica per archivi diversi

Una delle idee più eleganti di PowerShell sono i **provider**: adattatori che fanno apparire archivi di dati molto diversi (file, registro di Windows, certificati, variabili) come fossero **unità disco** navigabili con gli stessi comandi.

I provider integrati

Esempio 12.1 - elencare provider e relative unità logiche.

```
# Quali provider sono disponibili e quali 'unità' espongono:
Get-PSProvider
Get-PSDrive
```

Tabella 12.1 - i principali provider e le loro unità.

Provider	Unità (PSDrive)	Cosa espone
FileSystem	C:, D:, ...	File e cartelle
Registry	HKLM:, HKCU:	Registro di Windows
Certificate	Cert:	Certificati installati
Environment	Env:	Variabili d'ambiente
Variable	Variable:	Variabili di sessione
Function	Function:	Funzioni definite
Alias	Alias:	Alias definiti

Navigare archivi non-file come se fossero cartelle

La potenza sta qui: gli stessi `Get-ChildItem`, `Set-Location`, `Get-Item` funzionano su tutti i provider.

Esempio 12.2 - registro, ambiente e certificati con i comandi del file system.

```
# Navigare il registro di Windows come una cartella:
Set-Location HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion
Get-ChildItem | Select-Object -First 5 Name

# Leggere le variabili d'ambiente come 'file':
Get-ChildItem Env: | Where-Object Name -like 'PROCESSOR*'
(Get-Item Env:COMPUTERNAME).Value

# Elencare i certificati del computer:
Get-ChildItem Cert:\LocalMachine\My | Select-Object Subject, NotAfter
```

SUGGERIMENTO —

Questa uniformità è il sogno di chi gestisce sistemi eterogenei: è come avere un unico linguaggio per interrogare PLC, inverter e strumenti di fornitori diversi. Imparati i comandi una volta, valgono ovunque ci sia un provider.

Fonti

- Microsoft Learn, [about_Providers](https://learn.microsoft.com/powershell/about_providers), `Get-PSProvider`, `Get-PSDrive` - learn.microsoft.com/powershell

CAPITOLO 13

Lavorare con file e cartelle

La gestione di file e cartelle è il pane quotidiano dell'automazione: raccogliere log, archiviare report, organizzare backup. PowerShell la rende uniforme grazie ai cmdlet `*-Item` e `*-Content`.

Elencare e filtrare

Esempio 13.1 - `Get-ChildItem` con filtri di tipo e data.

```
# Elencare ricorsivamente solo i file .log più vecchi di 7 giorni:
Get-ChildItem -Path 'C:\Logs' -Filter '*.log' -Recurse -File |
    Where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-7) }

# Solo le cartelle, primo livello:
Get-ChildItem -Path 'D:\' -Directory
```

Creare, copiare, spostare, eliminare

Tabella 13.1 - i cmdlet `*-Item` per file e cartelle.

Operazione	Cmdlet	Esempio sintetico
Creare	<code>New-Item</code>	<code>New-Item -ItemType Directory -Path D:\bk</code>
Copiare	<code>Copy-Item</code>	<code>Copy-Item a.txt D:\bk\</code>
Spostare	<code>Move-Item</code>	<code>Move-Item a.txt D:\arch\</code>
Rinominare	<code>Rename-Item</code>	<code>Rename-Item a.txt b.txt</code>
Eliminare	<code>Remove-Item</code>	<code>Remove-Item a.txt -WhatIf</code>
Verificare	<code>Test-Path</code>	<code>Test-Path D:\bk\la.txt</code>

Esempio 13.2 - costruire percorsi con `Join-Path` (mai concatenare stringhe a mano).

```
# Creare una cartella di backup datata e copiarci dentro i file:
$dest = Join-Path 'D:\backup' (Get-Date -Format 'yyyy-MM-dd')
New-Item -ItemType Directory -Path $dest -Force | Out-Null
Copy-Item -Path 'C:\dati\*.csv' -Destination $dest -Verbose
```

ATTENZIONE —

`Remove-Item -Recurse -Force` è potente e **irreversibile**. Provarlo sempre prima con `-WhatIf`. La stessa prudenza che si usa prima di sovrascrivere il programma di un PLC in produzione.

Leggere e scrivere il contenuto

Esempio 13.3 - `Get-Content`, `Set-Content`, `Add-Content`.

```
# Leggere un file di testo (ogni riga è un elemento dell'array):
$righe = Get-Content -Path 'C:\Logs\система.log'
$righe.Count # quante righe
Get-Content 'C:\Logs\система.log' -Tail 20 # ultime 20 righe (come 'tail')

# Scrivere e accodare:
'Avvio procedura' | Set-Content -Path 'C:\Logs\proc.log' # crea/sovrascrive
'Passo 1 completato' | Add-Content -Path 'C:\Logs\proc.log' # accoda
```

Fonti

- Microsoft Learn, *Get-ChildItem*, *Copy-Item*, *Get-Content*, *Join-Path* - learn.microsoft.com/powershell

CAPITOLO 14

Stringhe, formattazione ed espressioni regolari

Elaborare testo - nomi di file, righe di log, messaggi - e costantemente necessario. PowerShell offre stringhe ricche, un operatore di formato e il supporto completo alle **espressioni regolari**.

Virgolette singole e doppie

Differenza fondamentale: gli apici **singoli** danno la stringa *letterale*; le virgolette **doppie** interpretano le variabili e le sequenze di escape.

Esempio 14.1 - apici singoli (letterali) contro doppi (interpolati).

```
$nome = 'Nord'
'Impianto $nome'      # -> Impianto $nome    (letterale, nessuna espansione)
"Impianto $nome"      # -> Impianto Nord      (la variabile viene espansa)

# Espressioni dentro la stringa con $( ):
"Totale file: $($((Get-ChildItem).Count))"
```

L'operatore di formato -f

Esempio 14.2 - allineamento e formattazione numerica/data con -f.

```
# Segnaposto {0}, {1}, ... con specificatori di formato:
'{0,-15} {1,8:N2} m3/h' -f 'Portata pompa', 42.5
'{0:yyyy-MM-dd HH:mm}' -f (Get-Date)
'{0:P1}' -f 0.873          # percentuale -> 87,3 %
```

Metodi delle stringhe e operatori di testo

Esempio 14.3 - metodi .NET delle stringhe e operatori -split / -join.

```
$s = ' Sensore-Livello-01 '
$s.Trim()          # rimuove gli spazi ai bordi
$s.ToUpper()       # MAIUSCOLO
$s.Replace('-', '_') # sostituzione semplice
'a,b,c' -split ',' # divide in array: a b c
('a','b','c') -join ' | ' # unisce: a | b | c
```

Espressioni regolari: -match e -replace

Le regex riconoscono schemi nel testo. **-match** verifica e popola la variabile automatica **\$Matches**; **-replace** sostituisce secondo uno schema.

Esempio 14.4 - regex per analizzare righe di log industriali.

```
# Estrarre un indirizzo IP da una riga di log:
$riga = '2026-05-01 ALLARME da 192.168.10.42 codice E12'
if ($riga -match '\d{1,3}(\.\d{1,3}){3}') { $Matches[0] } # 192.168.10.42

# Mascherare le ultime cifre di tutti gli IP in un testo:
$riga -replace '(\d+\.\d+\.\d+)\.\d+', '$1.xxx'

# Gruppi con nome per estrazioni leggibili:
if ($riga -match 'codice (?<cod>E\d+)') { $Matches.cod } # E12
```

SUGGERIMENTO —

Le espressioni regolari sono preziose per fare il *parsing* dei log esportati da PLC, SCADA o strumenti di campo, dove l'informazione utile è annegata nel testo. Vale la pena investirci: tornano utili in moltissimi contesti.

Fonti

- Microsoft Learn, *about_Quoting_Rules*, *about_Comparison_Operators*, *about_Regular_Expressions* - learn.microsoft.com/powershell

CAPITOLO 15

Array, collezioni e tabelle hash

Per gestire insiemi di dati servono le strutture: gli **array** (elenchi ordinati) e le **tabelle hash** (coppie chiave-valore). Sono alla base di configurazioni, mappature e accumulo di risultati.

Array

Esempio 15.1 - creazione, indici e operazioni sugli array.

```
$macchine = 'PLC1', 'PLC2', 'PLC3'      # array per enumerazione
$macchine[0]          # primo elemento -> PLC1
$macchine[-1]         # ultimo elemento -> PLC3
$macchine.Count        # quanti -> 3
$macchine += 'PLC4'    # aggiunge (crea un nuovo array)
$macchine -contains 'PLC2' # appartenenza -> True
1..10                  # intervallo: array da 1 a 10
```

NOTA —

Ogni `+=` su un array ne crea in realta uno nuovo copiando tutto: va benissimo per pochi elementi, ma su decine di migliaia diventa lento. In quei casi si usa `[System.Collections.Generic.List[object]]` con il metodo `.Add()`, molto piu efficiente.

Esempio 15.2 - List<T> quando l'array cresce molto.

```
# Lista efficiente per accumulare molti elementi in un ciclo:
$risultati = [System.Collections.Generic.List[object]]::new()
foreach ($n in 1..100000) { $risultati.Add($n * 2) }
$risultati.Count
```

Tabelle hash

Una **hashtable** associa chiavi a valori: perfetta per configurazioni e mappe di corrispondenza.

Esempio 15.3 - hashtable per parametri di configurazione.

```
$config = @{
    Server    = 'SRV-SCADA-01'
    Porta     = 502           # Modbus TCP
    Timeout   = 5
}
$config['Server']      # accesso per chiave
$config.Porta          # anche con la notazione punto
$config.Keys           # tutte le chiavi
$config['Retry'] = 3    # aggiunta di una nuova coppia
```

Una hashtable **ordinata** mantiene l'ordine di inserimento, utile quando lo si vuole prevedibile (ad esempio per costruire oggetti):

Esempio 15.4 - hashtable ordinata e conversione in PSCustomObject.

```
$ordinata = [ordered]@{ Primo = 1; Secondo = 2; Terzo = 3 }
# Trasformare una hashtable in oggetto strutturato:
[pscustomobject]$ordinata
```

Fonti

- Microsoft Learn, *about_Arrays*, *about_Hash_Tables* - learn.microsoft.com/powershell

CAPITOLO 16

Il controllo di flusso

Decisioni e ripetizioni trasformano una sequenza di comandi in un programma. PowerShell offre i costrutti classici, con la sintassi vista per gli operatori.

Condizioni: if / elseif / else e switch

Esempio 16.1 - if/elseif/else e switch con -Wildcard.

```
$liv = 78
if ($liv -ge 90)      { 'ALLARME ALTO' }
elseif ($liv -ge 70) { 'Attenzione' }
else                 { 'Normale' }

# switch: comodo per molti casi, supporta jolly e regex
switch -Wildcard ($codice) {
    'E*' { 'Errore' ; break }
    'W*' { 'Avviso' ; break }
    default { 'Sconosciuto' }
}
```

Cicli: for, foreach, while, do

Esempio 16.2 - i quattro tipi di ciclo.

```
# for: quando serve un contatore esplicito
for ($i = 1; $i -le 3; $i++) { "Tentativo $i" }

# foreach: iterare una collezione già in memoria
foreach ($m in 'PLC1','PLC2','PLC3') { Test-Connection $m -Count 1 -Quiet }

# while: finché la condizione è vera
$n = 0; while ($n -lt 3) { $n++; "ciclo $n" }

# do/while e do/until: il corpo viene eseguito almeno una volta
do { $r = Read-Host 'Continuare? (s/n)' } until ($r -eq 'n')
```

Interrompere e proseguire

break esce dal ciclo; **continue** salta all'iterazione successiva. Funzionano in tutti i cicli e nello **switch**.

Esempio 16.3 - break e continue in un ciclo di elaborazione file.

```
foreach ($f in Get-ChildItem 'C:\in' -Filter '*.dat') {
    if ($f.Length -eq 0) { continue }      # salta i file vuoti
    if ($f.Name -eq 'STOP.dat') { break }  # interrompe del tutto
    "Elaboro $($f.Name)"
}
```

SUGGERIMENTO —

La logica di flusso di PowerShell è concettualmente la stessa di un linguaggio strutturato come l'ST (Structured Text) della IEC 61131-3: chi programma PLC in ST ritroverà if, while e for con poche differenze di sintassi.

Fonti

- Microsoft Learn, [about_If](#), [about_Switch](#), [about_For](#), [about_Foreach](#), [about_While](#), [about_Do](#) - learn.microsoft.com/powershell

CAPITOLO 17

Funzioni, parametri e ambito (scope)

Le funzioni raggruppano comandi sotto un nome, per riusarli senza ripeterli. Sono il primo passo dal *comando usa-e-getta* allo *strumento riutilizzabile* - come passare da una sequenza scritta a mano a un blocco funzione di libreria nel PLC.

Definire una funzione

Esempio 17.1 - una funzione semplice senza parametri.

```
function Get-DiscoLibero {
    Get-PSDrive -PSProvider FileSystem |
        Select-Object Name,
            @{ N = 'LiberoGB'; E = { [math]::Round($_.Free / 1GB, 1) } }
}

Get-DiscoLibero          # si richiama come un cmdlet qualsiasi
```

Parametri con il blocco param

I parametri si dichiarano nel blocco `param()`, con il tipo e, facoltativamente, valori predefiniti e vincoli.

Esempio 17.2 - funzione con parametri tipizzati e valori predefiniti.

```
function Get-LogRecenti {
    param(
        [string]$Cartella = 'C:\Logs',
        [int]$Giorni = 7
    )
    $limite = (Get-Date).AddDays(-$Giorni)
    Get-ChildItem -Path $Cartella -Filter '*.log' -File |
        Where-Object LastWriteTime -ge $limite
}

Get-LogRecenti -Cartella 'D:\impianto\log' -Giorni 3
```

Funzioni avanzate: CmdletBinding e parametri obbligatori

Aggiungendo `[CmdletBinding()]` la funzione si comporta come un cmdlet vero: eredita i parametri comuni (`-Verbose`, `-WhatIf`...). Con gli attributi sui parametri si impongono regole.

Esempio 17.3 - funzione avanzata con Mandatory, ValidateSet e -WhatIf integrato.

```
function Stop-ServizioSicuro {
    [CmdletBinding(SupportsShouldProcess)]
    param(
        [Parameter(Mandatory, ValueFromPipeline)]
        [string]$Nome,
        [ValidateSet('Running', 'Stopped')]
        [string]$StatoAtteso = 'Running'
    )
    process {
        if ($PSCmdlet.ShouldProcess($Nome, 'Arresto servizio')) {
            Stop-Service -Name $Nome -ErrorAction Stop
        }
    }
}

# Ora supporta -WhatIf automaticamente, e accetta input dalla pipeline:
'Spooler' | Stop-ServizioSicuro -WhatIf
```


NOTA —

Mandatory rende il parametro obbligatorio (se manca, PowerShell lo chiede); **ValidateSet** ammette solo certi valori; **ValueFromPipeline** permette di ricevere l'input dalla pipeline. Sono i mattoni di uno strumento robusto.

L'ambito delle variabili (scope)

Una variabile creata dentro una funzione vive solo lì (ambito **locale**); non è visibile fuori, né modifica quelle esterne, salvo indicarlo esplicitamente. Questo isolamento previene effetti collaterali, esattamente come le variabili locali di un blocco funzione.

Esempio 17.4 - l'isolamento dell'ambito locale.

```
$globale = 'fuori'
function Prova {
    $locale = 'dentro'          # visibile solo qui
    $globale = 'modificata?'    # crea una COPIA locale, non tocca l'esterna
}
Prova
$globale    # ancora 'fuori': l'ambito locale ha protetto la variabile
```

Fonti

- Microsoft Learn, [about_Functions](#), [about_Functions_Advanced_Parameters](#), [about_Scopes](#) - learn.microsoft.com/powershell

CAPITOLO 18

La gestione degli errori

Uno script robusto non è quello che non incontra mai errori, ma quello che li **prevede e li gestisce**. Per chi lavora su impianti questo è ovvio: un automatismo serio contempla l'anomalia e reagisce in modo controllato, senza fermarsi a metà in stato indefinito.

Errori terminanti e non terminanti

PowerShell distingue gli errori **non terminanti** (segnalano un problema ma lo script prosegue) dai **terminanti** (interrompono l'esecuzione). Il parametro comune **-ErrorAction** controlla questo comportamento per ogni cmdlet.

Tabella 18.1 - valori di -ErrorAction.

Valore -ErrorAction	Effetto
Continue	Mostra l'errore e prosegue (predefinito)
Stop	Trasforma l'errore in terminante (attiva try/catch)
SilentlyContinue	Ignora l'errore e prosegue silenziosamente
Inquire	Chiede all'utente cosa fare
Ignore	Ignora e non registra nemmeno in \$Error

try / catch / finally

Il blocco **try** contiene il codice a rischio; **catch** gestisce l'errore se si verifica; **finally** viene eseguito **sempre**, utile per liberare risorse. Perché **catch** scatti, l'errore deve essere terminante: spesso si forza con **-ErrorAction Stop**.

Esempio 18.1 - try/catch con cattura specifica per tipo e catch generico.

```
try {
    $dati = Get-Content -Path 'D:\in\misure.csv' -ErrorAction Stop
    Write-Verbose "Lette $($dati.Count) righe"
}
catch [System.IO.FileNotFoundException] {
    Write-Warning 'File non trovato: salto questo ciclo'
}
catch {
    Write-Error "Errore imprevisto: $($_.Exception.Message)"
}
finally {
    Write-Verbose 'Blocco finally: eseguito comunque'
}
```

La variabile \$Error e throw

PowerShell conserva gli ultimi errori nell'array **\$Error** (il più recente in **\$Error[0]**). Con **throw** si solleva un errore terminante personalizzato, ad esempio quando un controllo di validità fallisce.

Esempio 18.2 - `$Error` e `throw` per validazioni.

```
$Error[0]          # l'ultimo errore avvenuto
$Error.Clear()     # svuota lo storico

function Set-Soglia {
    param([int]$Valore)
    if ($Valore -lt 0 -or $Valore -gt 100) {
        throw "Soglia $Valore fuori range (0-100)"
    }
    "Soglia impostata a $Valore"
}
Set-Soglia -Valore 150    # solleva l'errore
```

ATTENZIONE —

`-ErrorAction SilentlyContinue` è comodo ma pericoloso se abusato: nasconde i problemi. Meglio gestire l'errore con `try/catch` che ignorarlo. Un automatismo che 'tira dritto' su un'anomalia è spesso peggio di uno che si ferma in modo pulito segnalando il guasto.

Fonti

- Microsoft Learn, [about_Try_Catch_Finally](#), [about_CommonParameters](#), [about_Automatic_Variables](#) - learn.microsoft.com/powershell

CAPITOLO 19

Moduli e PowerShell Gallery

Un **modulo** è un pacchetto che raggruppa comandi su un tema: il modulo di Active Directory, quello di Hyper-V, quelli di terze parti. Permettono di estendere PowerShell senza reinventare nulla.

Trovare, importare e gestire i moduli

Esempio 19.1 - scoprire e importare i moduli.

```
# Moduli già disponibili sul sistema:
Get-Module -ListAvailable | Select-Object Name, Version

# Moduli caricati nella sessione corrente:
Get-Module

# Importare un modulo (spesso automatico al primo uso di un suo comando):
Import-Module -Name ActiveDirectory

# Quali comandi offre un modulo?
Get-Command -Module Microsoft.PowerShell.Archive
```

PowerShell Gallery: installare moduli di terze parti

La **PowerShell Gallery** è il repository pubblico ufficiale. Con **Find-Module** si cerca, con **Install-Module** si installa. Per l'utente corrente si usa **-Scope CurrentUser** (non richiede privilegi di amministratore).

Esempio 19.2 - gestione dei moduli dalla PowerShell Gallery.

```
# Cercare e installare un modulo dalla Gallery:
Find-Module -Name 'ImportExcel'
Install-Module -Name 'ImportExcel' -Scope CurrentUser

# Aggiornare o rimuovere:
Update-Module -Name 'ImportExcel'
Uninstall-Module -Name 'ImportExcel'
```

ATTENZIONE —

Installare moduli da Internet significa eseguire codice di terzi. Valutare la reputazione (download, autore, sorgente) e, in ambienti sensibili come le reti OT, preferire un repository **interno** approvato. La stessa cautela che si usa accettando una libreria di funzioni da un fornitore esterno.

NOTA —

I cmdlet più recenti per la gestione pacchetti appartengono al modulo *PowerShellGet* / *PSResourceGet* (es. **Install-PSResource**). La logica resta la stessa: cercare, installare con uno *scope*, aggiornare, rimuovere.

Fonti

- Microsoft Learn, *about_Modules*, *Install-Module*, *Get-Module* - learn.microsoft.com/powershell
- PowerShell Gallery - powershellgallery.com

CAPITOLO 20

Importare ed esportare dati: CSV, JSON, XML

Gli script raramente vivono isolati: leggono configurazioni, producono report, scambiano dati con altri sistemi. PowerShell converte nativamente tra oggetti e i formati più diffusi, mantenendo - dove possibile - la struttura.

CSV: lo scambio tabellare per eccellenza

Esempio 20.1 - Export-Csv / Import-Csv.

```
# Esportare oggetti in CSV (ideale per Excel e per i gestionali):
Get-Service | Select-Object Name, Status, StartType |
    Export-Csv -Path 'D:\report\servizi.csv' -NoTypeInfo -Encoding UTF8

# Importare un CSV: ogni riga diventa un oggetto con proprietà = intestazioni
$macchine = Import-Csv -Path 'D:\config\macchine.csv'
$macchine | Where-Object Reparto -eq 'Filtrazione' | Select-Object Nome, IP
```

JSON: configurazioni e API

Il **JSON** è oggi lo standard per configurazioni e per dialogare con le interfacce web (API REST). PowerShell converte avanti e indietro con due cmdlet.

Esempio 20.2 - ConvertTo-Json / ConvertFrom-Json.

```
# Da oggetto a JSON (per salvare una configurazione):
$cfg = [pscustomobject]@{ Server = 'SCADA01'; Porta = 502; Soglie = 70,90 }
$cfg | ConvertTo-Json | Set-Content -Path 'D:\config\impianto.json'

# Da JSON a oggetto (per leggerla e usarla):
$letto = Get-Content 'D:\config\impianto.json' -Raw | ConvertFrom-Json
$letto.Server      # SCADA01
$letto.Soglie[1]   # 90
```

NOTA —

Per JSON annidati profondi usare **ConvertTo-Json -Depth N**: il valore predefinito di profondità è basso e potrebbe troncare le strutture complesse, restituendo testo al posto degli oggetti interni.

XML e CliXml: fedeltà degli oggetti

L'**XML** resta diffuso in molti contesti industriali e gestionali. PowerShell offre inoltre il formato **CliXml**, che serializza oggetti PowerShell conservandone tipi e struttura - utile per salvare e ricaricare risultati tra sessioni.

Esempio 20.3 - Export/Import-Clixml e accesso a XML con [xml].

```
# CliXml: salva e ripristina oggetti mantenendone la struttura
Get-Process | Export-Clixml -Path 'D:\snap\processi.xml'
$ripristinati = Import-Clixml -Path 'D:\snap\processi.xml'

# Leggere un XML generico e navigarlo come oggetti:
[xml]$doc = Get-Content -Path 'D:\config\ricetta.xml' -Raw
$doc.ricetta.passo | Select-Object Numero, Durata
```

SUGGERIMENTO —

Per i tecnici dell'automazione questi cmdlet sono la colla tra mondi: un log esportato in CSV da uno SCADA, una ricetta in XML, una configurazione in JSON per un servizio - PowerShell li legge tutti e li trasforma in oggetti su cui ragionare con gli stessi comandi.

Fonti

- Microsoft Learn, *Import-Csv, ConvertTo-Json, Export-Clixml* - learn.microsoft.com/powershell
- Microsoft Learn, *What's New in PowerShell 7.5 (ConvertTo/From-CliXml)* - learn.microsoft.com/powershell

PARTE III

Sicurezza ed esecuzione degli script

Perche uno script scaricato da Internet non parte, come funziona davvero la execution policy, come firmare il codice e gestire credenziali e segreti. La sicurezza spiegata senza scorciatoie.

CAPITOLO 21

La execution policy in dettaglio

E forse la prima frustrazione di chi inizia: si scrive uno script, lo si lancia e PowerShell risponde che *l'esecuzione di script è disabilitata su questo sistema*. Non è un difetto: è la **execution policy**, un meccanismo pensato per evitare che codice indesiderato venga eseguito per sbaglio. Capirlo a fondo è indispensabile.

Che cosa è (e che cosa non è)

La execution policy decide **se e a quali condizioni** PowerShell accetta di eseguire file di script (.ps1). È importante chiarire subito un punto che Microsoft stessa sottolinea: **non è un confine di sicurezza**. Non serve a difendersi da un aggressore determinato - esistono molti modi legittimi di aggirarla - ma a impedire l'esecuzione *accidentale* di script, in particolare quelli arrivati da Internet.

ATTENZIONE —

Da ricordare bene: la execution policy **non protegge** da un utente malintenzionato. È un freno contro gli errori e le esecuzioni involontarie, non un sistema di sicurezza. Trattarla come tale sarebbe un grave errore di valutazione del rischio.

I tipi di execution policy

Esistono diversi livelli, dal più restrittivo al più permissivo:

Tabella 21.1 - i tipi di execution policy, dal più restrittivo al più permissivo.

Policy	Comportamento
Restricted	Nessuno script viene eseguito; si possono usare solo comandi interattivi. Predefinita sui client Windows.
AllSigned	Esegue solo script firmati digitalmente da un editore attendibile, anche quelli locali.
RemoteSigned	Gli script locali girano liberamente; quelli scaricati da Internet devono essere firmati. Predefinita su Windows Server.
Unrestricted	Esegue tutti gli script; per quelli da Internet mostra solo un avviso. Sconsigliata.
Bypass	Non blocca nulla e non avvisa. Pensata per scenari applicativi controllati.
Undefined	Nessuna policy impostata in questo ambito; vale quella dell'ambito successivo o la predefinita.

SUGGERIMENTO —

Per la maggior parte degli ambienti aziendali la scelta equilibrata è **RemoteSigned**: lascia lavorare gli script interni ma pretende una firma per quelli arrivati dall'esterno. È il compromesso più sensato tra praticità e prudenza.

Leggere e impostare la policy

Esempio 21.1 - Get-ExecutionPolicy e Set-ExecutionPolicy.

```
# Vedere la policy effettiva e quella per ciascun ambito:
Get-ExecutionPolicy           # quella attualmente in vigore
Get-ExecutionPolicy -List     # tutte, ambito per ambito

# Impostare RemoteSigned solo per l'utente corrente (non serve admin):
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

# Impostarla a livello di macchina (richiede privilegi di amministratore):
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope LocalMachine
```

Gli ambiti e la loro precedenza

La policy puo essere definita a piu **ambiti** (scope), che vengono valutati in ordine: il primo che risulta definito vince. Dal piu prioritario al meno prioritario:

1. **MachinePolicy** - imposta da criteri di gruppo (GPO) per tutta la macchina.
2. **UserPolicy** - imposta da GPO per l'utente.
3. **Process** - vale solo per la sessione corrente di PowerShell (non persistente).
4. **CurrentUser** - salvata nel registro per l'utente corrente.
5. **LocalMachine** - salvata nel registro per tutti gli utenti della macchina.

Le due policy imposte da criteri di gruppo (MachinePolicy e UserPolicy) hanno la precedenza assoluta e non sono modificabili localmente: in un dominio aziendale e spesso l'amministratore di rete a deciderle centralmente.

NOTA —

L'ambito **Process** e prezioso per i casi una-tantum: imposta la policy solo per la finestra corrente, senza lasciare traccia permanente. Si puo anche partire gia con una policy specifica: `powershell -ExecutionPolicy Bypass -File .\script.ps1`.

Perche uno script scaricato da Internet non parte

Con policy **RemoteSigned** (tipica) accade questo: uno script **creato localmente** gira senza problemi, ma uno **scaricato** (da web, e-mail, rete) viene bloccato se non e firmato. Come fa PowerShell a sapere che un file viene da Internet? Grazie al **Mark-of-the-Web**, descritto nel prossimo capitolo: Windows marchia i file scaricati, e PowerShell legge quel marchio.

Esempio 21.2 - il messaggio di blocco di uno script remoto non firmato.

```
# Esempio dell'errore tipico con uno script 'remoto' non firmato:
# .\installa.ps1 : File ...\installa.ps1 non puo essere caricato.
# Il file ...\installa.ps1 non e firmato digitalmente. Non e possibile
# eseguire questo script nel sistema corrente.
```

Come si aggira (e perche questo prova che non e sicurezza)

Proprio perche non e un confine di sicurezza, la policy si supera in molti modi del tutto documentati. Questo non e un trucco: e la prova che il meccanismo serve a prevenire l'errore, non l'attacco.

Esempio 21.3 - alcune vie legittime per oltrepassare la policy.

```
# Avviare un singolo script ignorando la policy, solo per quel processo:
powershell.exe -ExecutionPolicy Bypass -File 'C:\tmp\script.ps1'

# Eseguire il contenuto di uno script senza che sia un file .ps1:
Get-Content .\script.ps1 -Raw | Invoke-Expression # (sconsigliato)

# La policy si applica ai FILE .ps1, non ai comandi digitati o incollati.
```

ATTENZIONE —

Invoke-Expression esegue testo arbitrario come codice: e potenzialmente pericoloso e va evitato, specie su input che non controlliamo. E mostrato qui solo per spiegare perché la execution policy non possa essere considerata una difesa.

Fonti

- Microsoft Learn, *about_Execution_Policies* - learn.microsoft.com/powershell
- Microsoft Learn, *Set-ExecutionPolicy*, *Get-ExecutionPolicy* - learn.microsoft.com/powershell

CAPITOLO 22

Mark-of-the-Web: i file che arrivano da fuori

Il **Mark-of-the-Web** (MOTW) è il meccanismo con cui Windows ricorda che un file proviene da una fonte esterna. E la chiave per capire perché un certo `.ps1` viene bloccato e un altro no.

Come Windows marchia i file scaricati

Quando un browser o un client di posta salva un file proveniente da Internet, Windows aggiunge al file un **flusso di dati alternativo** (ADS) chiamato `Zone.Identifier`, che indica la *zona* di origine. La zona 3 corrisponde a Internet. PowerShell, con policy `RemoteSigned` o `AllSigned`, legge questo marchio e tratta il file come remoto.

Esempio 22.1 - ispezionare il flusso `Zone.Identifier` (Mark-of-the-Web).

```
# Vedere il marchio di zona di un file scaricato:
Get-Content -Path '.\installer.ps1' -Stream Zone.Identifier
# [ZoneTransfer]
# ZoneId=3 <-- 3 = Internet

# Elencare gli stream alternativi di un file:
Get-Item -Path '.\installer.ps1' -Stream *
```

Sbloccare un file con Unblock-File

Se ci si fida della provenienza, si rimuove il marchio con `Unblock-File`. Da quel momento lo script è trattato come locale.

Esempio 22.2 - `Unblock-File` per rimuovere il Mark-of-the-Web.

```
# Sbloccare un singolo file dopo averne verificato la provenienza:
Unblock-File -Path '.\installer.ps1'

# Sbloccare in blocco tutti i .ps1 di una cartella (con cautela!):
Get-ChildItem '.\scripts' -Filter '*.ps1' -Recurse | Unblock-File
```

ATTENZIONE —

Sbloccare un file significa dichiarare che ci si fida del suo contenuto. Va fatto solo dopo averlo **letto e compreso**. Nelle reti industriali, dove un'azione sbagliata ha conseguenze fisiche, questa verifica è doppiamente importante.

SUGGERIMENTO —

Per chi viene dall'automazione: il Mark-of-the-Web è come l'etichetta *materiale non collaudato* su un componente appena arrivato dal fornitore. Non lo monti in linea finché qualcuno non lo verifica e ne autorizza l'uso.

Fonti

- Microsoft Learn, *Unblock-File* - learn.microsoft.com/powershell
- Microsoft Learn, *about_Execution_Policies* (sezione Mark of the Web) - learn.microsoft.com/powershell

CAPITOLO 23

Firmare digitalmente gli script

La firma digitale risolve il problema alla radice: uno script firmato da un editore attendibile e accettato anche con le policy più severe (**AllSigned**), e qualunque manomissione successiva invalida la firma. E il modo professionale di distribuire script in azienda.

Il principio

Firmare uno script significa applicargli una *firma crittografica* ottenuta con un **certificato di firma del codice** (*code signing*). Chi esegue lo script può verificare due cose: che provenga davvero da quell'editore e che non sia stato modificato dopo la firma. E lo stesso principio di integrità che si vorrebbe avere sul firmware di un dispositivo di campo.

Firmare e verificare

Esempio 23.1 - firmare con `Set-AuthenticodeSignature` e verificare.

```
# Individuare un certificato di code-signing disponibile:
$cert = Get-ChildItem Cert:\CurrentUser\My -CodeSigningCert | Select-Object -First 1

# Applicare la firma allo script:
Set-AuthenticodeSignature -FilePath '.\deploy.ps1' -Certificate $cert

# Verificare la firma di uno script:
Get-AuthenticodeSignature -FilePath '.\deploy.ps1' |
    Select-Object Status, SignerCertificate
```

NOTA —

In produzione il certificato dovrebbe provenire da una **autorità di certificazione** attendibile (interna all'azienda o pubblica), non essere autofirmato. Un certificato autofirmato è utile solo per prove in laboratorio.

Con policy **AllSigned**, anche un piccolo script locale deve essere firmato: massima sicurezza, ma anche massima rigidità. La maggior parte delle organizzazioni preferisce **RemoteSigned**, riservando **AllSigned** agli ambienti più critici.

Fonti

- Microsoft Learn, *Set-AuthenticodeSignature*, *Get-AuthenticodeSignature* - learn.microsoft.com/powershell
- Microsoft Learn, *about_Signing* - learn.microsoft.com/powershell

CAPITOLO 24

Credenziali e segreti

Gli script spesso devono autenticarsi: collegarsi a un server, a un database, a un'API. La regola d'oro è una sola: **mai scrivere password in chiaro** dentro gli script. PowerShell offre strumenti appositi per gestire le credenziali in modo sicuro.

PSCredential e Get-Credential

L'oggetto **PSCredential** rappresenta una coppia utente/password in cui la password è custodita in forma protetta (**SecureString**). Si ottiene interattivamente con **Get-Credential**.

Esempio 24.1 - ottenere e usare un oggetto PSCredential.

```
# Chiede utente e password con una finestra sicura, senza mostrarli:
$cred = Get-Credential -Message 'Credenziali per il server SCADA'

# Molti cmdlet accettano direttamente -Credential:
Invoke-Command -ComputerName 'SRV-SCADA-01' -Credential $cred -ScriptBlock {
    Get-Service -Name 'W3SVC'
}
```

SecureString: cosa è e cosa non è

Un **SecureString** mantiene il testo cifrato in memoria, riducendo il rischio di esposizione accidentale. Attenzione però: protegge il dato *a riposo* sulla stessa macchina e con lo stesso utente; non è una cassaforte universale né un sistema di trasporto sicuro tra computer diversi.

Esempio 24.2 - persistere e ricaricare una password protetta.

```
# Salvare una password protetta su file (utile per attività pianificate):
# (decifrabile SOLO dallo stesso utente sulla stessa macchina)
Read-Host 'Password' -AsSecureString | ConvertFrom-SecureString |
    Set-Content -Path 'C:\segreti\db.sec'

# Rileggerla in uno script automatico:
$sec = Get-Content 'C:\segreti\db.sec' | ConvertTo-SecureString
$cred = [System.Management.Automation.PSCredential]::new('svc_db', $sec)
```

ATTENZIONE —

Il file prodotto da **ConvertFrom-SecureString** è legato a utente e macchina: se copiato altrove non sarà decifrabile. È un pregio (non è riutilizzabile da altri) ma anche un vincolo da pianificare nelle attività automatiche.

Il modulo SecretManagement

La soluzione moderna è il modulo **Microsoft.PowerShell.SecretManagement**, che offre un'interfaccia unica verso *vault* di segreti (locali o aziendali, come gestori dedicati). Lo script chiede il segreto per nome, senza mai contenerlo.

Esempio 24.3 - SecretManagement: i segreti vivono nel vault, non nello script.

```
# Registrare un vault locale e usarlo (richiede i moduli SecretManagement):
Register-SecretVault -Name LocaleVault -ModuleName Microsoft.PowerShell.SecretStore -DefaultVault
Set-Secret -Name 'Db/Prod' -Secret 'la-mia-password'

# Negli script si recupera per NOME, senza scrivere il segreto:
$pwd = Get-Secret -Name 'Db/Prod' -AsPlainText
```

SUGGERIMENTO —

In un contesto industriale, centralizzare i segreti in un vault è l'equivalente di tenere le chiavi del quadro elettrico in una bacheca controllata invece che appese a ogni macchina: meno copie in giro, accesso tracciato, revoca semplice.

Fonti

- Microsoft Learn, *Get-Credential, ConvertTo-SecureString* - learn.microsoft.com/powershell
- Microsoft Learn, *SecretManagement and SecretStore* - learn.microsoft.com/powershell/utility-modules/secretmanagement

CAPITOLO 25

Logging e funzionalita di sicurezza avanzate

Negli ambienti gestiti, PowerShell può essere configurato per **registrare** cosa viene eseguito e per **limitare** cosa e permesso. Sono funzioni che interessano soprattutto chi amministra reti aziendali, comprese le reti OT, dove tracciabilità e controllo sono requisiti, spesso normativi.

Registrazione: transcription e script block logging

- **Transcription** - salva su file una trascrizione completa delle sessioni (comandi e output). Si attiva via criteri di gruppo o con **Start-Transcript**.
- **Module logging** - registra le pipeline eseguite dai moduli indicati.
- **Script Block Logging** - registra nel log eventi i blocchi di codice eseguiti, anche se offuscati: prezioso per le indagini di sicurezza.

Esempio 25.1 - Start-Transcript per documentare una sessione.

```
# Avviare manualmente una trascrizione della sessione corrente:
Start-Transcript -Path 'C:\Logs\sessione_$(Get-Date -Format yyyyMMdd_HH:mm).txt'
# ... comandi ...
Stop-Transcript
```

AMSI: l'analisi antimalware in tempo reale

L'**Antimalware Scan Interface** (AMSI) consente all'antivirus di ispezionare il codice che PowerShell sta per eseguire, anche quando viene generato dinamicamente in memoria. È un controllo a livello di sistema, trasparente per l'utente, che intercetta script sospetti prima dell'esecuzione.

Constrained Language Mode e JEA

Per ridurre la superficie di rischio esistono due strumenti potenti:

- **Constrained Language Mode** - limita il linguaggio a un sottoinsieme sicuro, impedendo l'accesso arbitrario alle classi .NET e alle API più delicate.
- **Just Enough Administration (JEA)** - permette di concedere a un operatore solo i comandi che gli servono, tramite *endpoint* vincolati: l'utente può fare esattamente quelle operazioni e nient'altro, anche senza essere amministratore.

SUGGERIMENTO —

JEA è l'equivalente, nel mondo IT, dei livelli di accesso di un pannello HMI: l'operatore di linea vede e comanda solo ciò che gli compete, mentre le funzioni di manutenzione restano protette. Il principio è lo stesso: **privilegio minimo**.

NOTA —

La sicurezza di PowerShell è a strati: execution policy contro gli errori, firma per l'integrità, gestione protetta dei segreti, logging per la tracciabilità, AMSI per l'analisi, JEA e Constrained Language per il privilegio minimo. Nessuno strato basta da solo: insieme costituiscono una difesa solida.

Fonti

- Microsoft Learn, *about_Logging*, *Start-Transcript* - learn.microsoft.com/powershell
- Microsoft Learn, *PowerShell security features* e *Just Enough Administration (JEA)* - learn.microsoft.com/powershell/scripting/security

PARTE IV

Scripting: dal semplice al complesso

Dal primo file .ps1 alle funzioni avanzate, alle classi e ai moduli riutilizzabili. Costruiamo strumenti solidi, passo dopo passo, con lo stesso rigore che si applica al codice di un automatismo.

CAPITOLO 26

Il primo script

Uno **script** non è altro che un file di testo con estensione **.ps1** che contiene comandi PowerShell, eseguiti dall'alto verso il basso. Passare dalla console allo script è come passare da un comando manuale a una sequenza memorizzata nel PLC: la stessa logica, ma ripetibile e documentata.

Struttura di base

Esempio 26.1 - un primo script lineare, leggibile e commentato.

```
# === backup-log.ps1 =====
# Scopo : copia i log del giorno in una cartella di archivio datata
# Autore: Reparto automazione
# =====

$origine = 'C:\impianto\log'
$archivio = 'D:\archivio\log'
$soggi = Get-Date -Format 'yyyy-MM-dd'
$destinazione = Join-Path $archivio $soggi

New-Item -ItemType Directory -Path $destinazione -Force | Out-Null
Copy-Item -Path (Join-Path $origine '*.log') -Destination $destinazione

Write-Host "Backup completato in $destinazione"
```

Eseguire uno script

Per eseguire uno script si indica il suo percorso. Se si trova nella cartella corrente, va anteposto **.** (PowerShell, per sicurezza, non cerca gli eseguibili nella cartella corrente, a differenza del vecchio DOS).

Esempio 26.2 - modi di avviare uno script.

```
.\backup-log.ps1           # script nella cartella corrente
C:\strumenti\backup-log.ps1 # percorso assoluto

# Se la execution policy blocca, ricordare quanto visto nella Parte III:
powershell -ExecutionPolicy Bypass -File .\backup-log.ps1
```

NOTA —

Il prefisso **.** non è un capriccio: impedisce che, digitando il nome di uno script, venga eseguito per errore un file con lo stesso nome capitato nella cartella corrente. È una piccola difesa contro un classico inganno.

Commenti e aiuto integrato

Oltre ai commenti con **#**, esiste il **commento basato su aiuto** (*comment-based help*): blocchi speciali che **Get-Help** riconosce, trasformando il proprio script in uno strumento documentato come un cmdlet ufficiale.

Esempio 26.3 - comment-based help: il proprio script entra in Get-Help.

```
<#  
.SYNOPSIS  
    Archivia i file di log del giorno.  
.DESCRIPTION  
    Copia i .log dalla cartella di impianto a un archivio datato.  
.PARAMETER Origine  
    Cartella sorgente dei log.  
.EXAMPLE  
    .\backup-log.ps1 -Origine 'C:\impianto\log'  
#>  
param([string]$Origine = 'C:\impianto\log')
```

Fonti

- Microsoft Learn, *about_Scripts* - learn.microsoft.com/powershell
- Microsoft Learn, *about_Comment_Based_Help* - learn.microsoft.com/powershell

CAPITOLO 27

Input, output e parametrizzazione

Uno script utile è uno script **parametrico**: non valori fissi nel codice, ma parametri che ne adattano il comportamento. E gli output vanno distinti per canale, esattamente come si separano i segnali di processo dagli allarmi.

Parametri robusti

Esempio 27.1 - parametri con obbligatorieta e vincoli di validita.

```
param(  
    [Parameter(Mandatory)]  
    [string]$Cartella,  
  
    [ValidateRange(1, 365)]  
    [int]$GiorniRitenzione = 30,  
  
    [ValidateSet('Copia', 'Sposta')]  
    [string]$Modalita = 'Copia',  
  
    [switch]$Simula  
)
```

Gli attributi di validazione (`ValidateRange`, `ValidateSet`, `ValidateNotNullOrEmpty`, ...) bloccano i valori sbagliati **prima** che lo script faccia danni: e il controllo di plausibilita degli ingressi, lo stesso che un buon programma PLC fa sui set-point.

I canali di output

PowerShell non ha un solo output: ne ha diversi, ciascuno con uno scopo. Confonderli è un errore comune.

Tabella 27.1 - i canali di output e quando usarli.

Cmdlet	Canale	Uso corretto
Write-Output	Output (1)	Dati 'veri' che proseguono nella pipeline
Write-Verbose	Verbose (4)	Dettagli diagnostici (visibili con -Verbose)
Write-Warning	Warning (3)	Avvisi non bloccanti
Write-Error	Error (2)	Errori non terminanti
Write-Host	Schermo	Messaggi all'utente (NON è output di dati)
Write-Progress	Barra	Avanzamento di operazioni lunghe

ATTENZIONE —

`Write-Host` scrive a schermo ma **non** restituisce dati nella pipeline: non usarlo per produrre risultati da elaborare. Per i dati si usa `Write-Output` (o si lascia semplicemente 'cadere' l'oggetto). Confondere i due è all'origine di molti script che 'funzionano a video ma non passano nulla' al comando successivo.

Esempio 27.2 - Write-Progress per le operazioni lunghe.

```
# Mostrare l'avanzamento di un ciclo lungo:
$file = Get-ChildItem 'C:\in' -Filter '*.dat'
for ($i = 0; $i -lt $file.Count; $i++) {
    Write-Progress -Activity 'Elaborazione' -Status $file[$i].Name `
        -PercentComplete (($i + 1) / $file.Count * 100)
    Start-Sleep -Milliseconds 50      # simulazione lavoro
}
```

Fonti

- Microsoft Learn, *about_Functions_Advanced_Parameters* - learn.microsoft.com/powershell
- Microsoft Learn, *Write-Output*, *Write-Verbose*, *Write-Progress* - learn.microsoft.com/powershell

CAPITOLO 28

Cicli, file e log in pratica

Mettiamo insieme i pezzi in uno scenario realistico: uno script che elabora una serie di file, registra cosa fa in un log strutturato e non si ferma al primo intoppo. E il cuore di moltissime automazioni di back-office industriale.

Un log strutturato e riutilizzabile

Esempio 28.1 - una funzione di logging con timestamp e livello.

```
function Write-Log {  
    param(  
        [Parameter(Mandatory)][string]$Messaggio,  
        [ValidateSet('INFO','WARN','ERR')][string]$Livello = 'INFO',  
        [string]$File = 'C:\Logs\elaborazione.log'  
    )  
    $riga = '{0:yyyy-MM-dd HH:mm:ss} [{1}] {2}' -f (Get-Date), $Livello, $Messaggio  
    Add-Content -Path $File -Value $riga  
    if ($Livello -eq 'ERR') { Write-Warning $Messaggio }  
}
```

Il ciclo di elaborazione robusto

La struttura tipica: si scorrono i file, ogni elaborazione è protetta da `try/catch`, si registra l'esito e si tiene un contatore. Un errore su un file non deve fermare gli altri.

Esempio 28.2 - elaborazione a prova di errore con logging dell'esito.

```
$elaborati = 0; $falliti = 0  
Write-Log -Messaggio 'Avvio elaborazione batch'  
  
foreach ($f in Get-ChildItem 'C:\in' -Filter '*.csv' -File) {  
    try {  
        $dati = Import-Csv -Path $f.FullName -ErrorAction Stop  
        # ... elaborazione dei dati ...  
        Write-Log -Messaggio "OK $($f.Name): $($dati.Count) righe"  
        $elaborati++  
    }  
    catch {  
        Write-Log -Messaggio "FALLITO $($f.Name): $($_.Exception.Message)" -Livello ERR  
        $falliti++  
    }  
}  
  
Write-Log -Messaggio "Fine batch: $elaborati ok, $falliti falliti"
```

SUGGERIMENTO —

Questo schema - scorri, proteggi ogni passo, registra l'esito, conta - è direttamente trasferibile all'automazione: è la stessa logica di una sequenza che elabora i pezzi su una linea, segnala lo scarto e prosegue senza bloccare la produzione.

Fonti

- Microsoft Learn, *about_Try_Catch_Finally*, *Add-Content* - learn.microsoft.com/powershell

CAPITOLO 29

Funzioni avanzate e pipeline

Le funzioni più potenti accettano input dalla **pipeline** ed emettono **oggetti**, integrandosi con tutto il resto di PowerShell. Per farlo si usa la struttura a tre blocchi **begin** / **process** / **end**.

Begin, process, end

Quando una funzione riceve oggetti dalla pipeline, il blocco **process** viene eseguito **una volta per ciascun oggetto** in arrivo; **begin** una sola volta all'inizio (per inizializzare), **end** una sola volta alla fine (per concludere). È il modo corretto di scrivere funzioni che 'scorrono' i dati.

Esempio 29.1 - funzione che elabora la pipeline elemento per elemento.

```
function Convert-InMaiuscolo {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory, ValueFromPipeline)]
        [string]$Testo
    )
    begin { $n = 0 }
    process { $n++; $Testo.ToUpper() }      # eseguito per ogni elemento
    end { Write-Verbose "Elaborate $n stringhe" }
}

'plc1','plc2','plc3' | Convert-InMaiuscolo -Verbose
```

Ricevere proprietà dagli oggetti

Con **ValueFromPipelineByPropertyName** la funzione può agganciare automaticamente le proprietà degli oggetti in arrivo ai propri parametri omonimi: così si concatena con cmdlet che producono oggetti ricchi.

Esempio 29.2 - agganciare le proprietà della pipeline ai parametri.

```
function Test-Macchina {
    param(
        [Parameter(Mandatory, ValueFromPipelineByPropertyName)]
        [string]$Nome,
        [Parameter(ValueFromPipelineByPropertyName)]
        [string]$IP
    )
    process {
        [pscustomobject]@{
            Nome      = $Nome
            IP         = $IP
            Raggiungibile = Test-Connection $IP -Count 1 -Quiet
        }
    }
}

# Un CSV con colonne 'Nome' e 'IP' alimenta direttamente la funzione:
Import-Csv 'C:\config\macchine.csv' | Test-Macchina | Format-Table -AutoSize
```

NOTA —

Emettere **oggetti** ([pscustomobject]) invece di stringhe e la chiave per scrivere strumenti componibili: l'output della propria funzione si potrà ordinare, filtrare, esportare in CSV o JSON, esattamente come quello dei cmdlet nativi. È il principio della pipeline a oggetti applicato al proprio codice.

Fonti

- Microsoft Learn, *about_Functions_Advanced*, *about_Functions_Advanced_Methods* - learn.microsoft.com/powershell
- Microsoft Learn, *about_Pipelines* - learn.microsoft.com/powershell

CAPITOLO 30

Le classi in PowerShell

Dalla versione 5.0 PowerShell supporta le **classi**: il modo per definire propri tipi di oggetto, con proprietà e metodi. Utili quando si modellano entità del proprio dominio - una macchina, una ricetta, un allarme - in modo strutturato.

Definire una classe

Esempio 30.1 - una classe con proprietà, costruttore e metodo.

```
class Macchina {
    [string]$Nome
    [string]$IP
    [int]$Porta = 502          # Modbus TCP come valore predefinito

    # Costruttore
    Macchina([string]$nome, [string]$ip) {
        $this.Nome = $nome
        $this.IP    = $ip
    }

    # Metodo
    [bool] ERaggiungibile() {
        return Test-Connection $this.IP -Count 1 -Quiet
    }
}

$m = [Macchina]::new('PLC-Filtri', '192.168.10.20')
$m.Porta
$m.ERaggiungibile()
```

Enumerazioni ed ereditarietà

Le **enum** danno nomi leggibili a insiemi di valori; l'**ereditarietà** permette a una classe di specializzarne un'altra.

Esempio 30.2 - enum ed ereditarietà tra classi.

```
enum StatoMacchina { Ferma; InMarcia; Allarme }

class Pompa : Macchina {          # eredita da Macchina
    [StatoMacchina]$Stato = [StatoMacchina]::Ferma
    [double]$PortataNominale

    [string] Descrivi() {
        return '{0} ({1}) - {2}' -f $this.Nome, $this.IP, $this.Stato
    }
}

$p = [Pompa]::new('Pompa-1', '192.168.10.31')
$p.Stato = [StatoMacchina]::InMarcia
$p.Descrivi()
```

NOTA —

Le classi PowerShell sono comode per modellare dati e per scenari che richiedono tipi strutturati (ad esempio risorse DSC). Per la maggior parte degli script, però, restano più idiomatiche le funzioni che emettono `[pscustomobject]`: meglio non complicare ciò che può restare semplice.

Fonti

- Microsoft Learn, *about_Classes*, *about_Enum* - learn.microsoft.com/powershell

CAPITOLO 31

Creare un modulo riutilizzabile

Quando un insieme di funzioni si stabilizza, conviene impacchettarlo in un **modulo**: così si carica con `Import-Module`, si versiona e si distribuisce. E il salto dallo script personale alla libreria condivisa dal reparto.

Il file di modulo .psm1

Un modulo, nella forma più semplice, è un file `.psm1` che contiene le funzioni e dichiara quali rendere pubbliche con `Export-ModuleMember`.

Esempio 31.1 - un modulo .psm1 con funzioni esportate.

```
# === ImpiantoTools.psm1 =====
function Get-StatoServizi {
    param([string[]]$Nome = @('W3SVC','MSSQLSERVER'))
    Get-Service -Name $Nome -ErrorAction SilentlyContinue |
        Select-Object Name, Status
}

function Test-ReteImpianto {
    param([Parameter(Mandatory)][string[]]$IP)
    foreach ($a in $IP) {
        [pscustomobject]{ IP = $a; Su = Test-Connection $a -Count 1 -Quiet }
    }
}

# Rende pubbliche SOLO le funzioni previste:
Export-ModuleMember -Function Get-StatoServizi, Test-ReteImpianto
```

Il manifesto .psd1

Per i moduli seri si affianca un **manifesto** (`.psd1`): un file di metadati che descrive versione, autore, dipendenze e cosa esportare. È il documento d'identità del modulo - prezioso per la manutenzione e per la gestione delle versioni.

Esempio 31.2 - New-ModuleManifest crea il file .psd1.

```
# Generare un manifesto di base:
New-ModuleManifest -Path '.\ImpiantoTools.psd1' `
    -RootModule 'ImpiantoTools.psm1' `
    -ModuleVersion '1.0.0' `
    -Author 'Reparto Automazione' `
    -Description 'Utility per il monitoraggio dell'impianto' `
    -FunctionsToExport @('Get-StatoServizi','Test-ReteImpianto')
```

Perché PowerShell trovi il modulo con il solo nome, la cartella che lo contiene va collocata in uno dei percorsi elencati nella variabile `$env:PSModulePath`. La convenzione è una cartella con il nome del modulo, contenente `.psd1` e `.psm1`.

Esempio 31.3 - posizionare e importare il modulo per nome.

```
# Dove PowerShell cerca i moduli:
$env:PSModulePath -split ';'

# Struttura consigliata:
# ... \Modules\ImpiantoTools\ImpiantoTools.psd1
# ... \Modules\ImpiantoTools\ImpiantoTools.psm1

Import-Module ImpiantoTools
Get-Command -Module ImpiantoTools
```

SUGGERIMENTO —

Versionare il modulo nel manifesto (**ModuleVersion**) e l'equivalente del numero di revisione di un programma PLC: permette di sapere con esattezza quale versione gira su una macchina e di gestire gli aggiornamenti in modo controllato e tracciabile.

Fonti

- Microsoft Learn, *about_Modules*, *New-ModuleManifest*, *Export-ModuleMember* - learn.microsoft.com/powershell

CAPITOLO 32

Qualita, stile e test

Uno script che gira non basta: deve essere **leggibile, robusto e verificabile**. Chi lavora in ambito industriale conosce il valore delle buone pratiche e dei collaudi; valgono anche qui.

Buone pratiche di stile

- Usare la convenzione **Verbo-Nome** con verbi approvati per le proprie funzioni.
- Parametri nominali, tipizzati e validati; niente valori 'magici' nascosti nel codice.
- Una funzione = un compito chiaro. Funzioni piccole e componibili.
- Emettere **oggetti**, non testo, così l'output è riutilizzabile.
- Gestire gli errori con `try/catch`; non silenziare i problemi.
- Documentare con il **comment-based help** (`.SYNOPSIS`, `.EXAMPLE`).
- Indentazione coerente e nomi parlanti, in una sola lingua.

PSScriptAnalyzer: il controllo automatico dello stile

Il modulo **PSScriptAnalyzer** analizza il codice e segnala errori comuni, anti-pattern e violazioni di stile: una sorta di revisore automatico, paragonabile al controllo statico del codice nei tool di sviluppo industriale.

Esempio 32.1 - analisi statica con PSScriptAnalyzer.

```
Install-Module PSScriptAnalyzer -Scope CurrentUser
Invoke-ScriptAnalyzer -Path '.\backup-log.ps1'
Invoke-ScriptAnalyzer -Path '.\src' -Recurse -Severity Warning,Error
```

Pester: scrivere test automatici

Pester è il framework di test di PowerShell. Permette di scrivere verifiche che controllano automaticamente che le funzioni si comportino come previsto - il collaudo del software, da rieseguire a ogni modifica.

Esempio 32.2 - un test Pester con Describe/It/Should.

```
# salvataggio: ImpiantoTools.Tests.ps1
Describe 'Test-ReteImpianto' {
    It 'restituisce un oggetto per ogni IP fornito' {
        $r = Test-ReteImpianto -IP '127.0.0.1','127.0.0.2'
        $r.Count | Should -Be 2
    }
    It 'ogni risultato ha la proprietà Su' {
        $r = Test-ReteImpianto -IP '127.0.0.1'
        $r[0].PSObject.Properties.Name | Should -Contain 'Su'
    }
}

# Esecuzione dei test:
Invoke-Pester -Path '.\ImpiantoTools.Tests.ps1'
```

SUGGERIMENTO —

Un test automatico è come un collaudo funzionale che si può ripetere a costo zero: ogni volta che si modifica uno strumento, si rilanciano i test e si ha la certezza di non aver rotto ciò che già funzionava. Inestimabile sugli script che governano processi reali.

-
- PSScriptAnalyzer - github.com/PowerShell/PSScriptAnalyzer ; Microsoft Learn
 - Pester - pester.dev ; Microsoft Learn, *Testing with Pester*

CAPITOLO 33

Attività pianificate e automazione headless

Il vero valore dell'automazione si esprime quando gli script girano **da soli**, su pianificazione, senza nessuno alla tastiera. Su Windows questo si fa con l'**Utilità di pianificazione** (Task Scheduler), interamente governabile da PowerShell.

Creare un'attività pianificata da PowerShell

Si compone l'attività da tre elementi: l'**azione** (cosa eseguire), il **trigger** (quando) e le impostazioni, poi la si registra.

Esempio 33.1 - pianificare uno script con Register-ScheduledTask.

```
# Azione: eseguire il nostro script di backup ogni notte alle 23:00
$action = New-ScheduledTaskAction -Execute 'pwsh.exe' `
    -Argument '-NoProfile -File C:\strumenti\backup-log.ps1'

$trigger = New-ScheduledTaskTrigger -Daily -At '23:00'

$impost = New-ScheduledTaskSettingsSet -StartWhenAvailable `
    -RunOnlyIfNetworkAvailable

Register-ScheduledTask -TaskName 'Backup log impianto' `
    -Action $action -Trigger $trigger -Settings $impost `
    -Description 'Archiviazione notturna dei log'
```

Gestire e verificare le attività

Esempio 33.2 - ciclo di vita di un'attività pianificata.

```
# Elencare, ispezionare lo storico, avviare a mano, rimuovere:
Get-ScheduledTask -TaskName 'Backup*'
Get-ScheduledTaskInfo -TaskName 'Backup log impianto' # ultimo esito
Start-ScheduledTask -TaskName 'Backup log impianto' # esecuzione manuale
Unregister-ScheduledTask -TaskName 'Backup log impianto' -Confirm:$false
```

Scrivere script adatti all'esecuzione non presidiata

- Avviare con **-NoProfile** per non dipendere dal profilo utente, e **-NonInteractive** per evitare richieste a video.
- **Mai** prevedere input interattivi (**Read-Host**) in uno script pianificato: bloccherebbe l'esecuzione.
- Registrare sempre un **log** dell'esito (come nel capitolo 28): senza nessuno a guardare, il log è l'unico modo di sapere cosa è successo.
- Restituire codici di uscita coerenti con **exit** per segnalare successo o errore a chi pianifica.
- Gestire le credenziali con i metodi sicuri della Parte III, mai in chiaro.

Esempio 33.3 - exit code per segnalare l'esito a chi pianifica.

```
# Schema di uno script 'headless' che comunica l'esito con un exit code:
try {
    # ... lavoro ...
    Write-Log 'Attività completata'
    exit 0 # 0 = successo
}
catch {
    Write-Log "Errore: $($_.Exception.Message)" -Livello ERR
    exit 1 # diverso da 0 = errore
}
```

SUGGERIMENTO —

Un'attività pianificata che archivia log, controlla servizi o raccoglie misure di notte e l'esatto parallelo di un automatismo ciclico in campo: gira da solo, deve essere affidabile e deve lasciare traccia di ciò che fa. La differenza è che qui il 'campo' è il file system e la rete.

Fonti

- Microsoft Learn, *ScheduledTasks* module (*Register-ScheduledTask*, *New-ScheduledTaskTrigger*) - learn.microsoft.com/powershell
- Microsoft Learn, *about_Automatic_Variables* (codici di uscita) - learn.microsoft.com/powershell

PARTE V

Amministrare Windows Server con PowerShell

Fare con i comandi tutto ciò che si farebbe con l'interfaccia grafica: gestione remota, ruoli, rete, Active Directory, storage, servizi ed eventi. Perché sui server la riga di comando non è un'alternativa, ma la norma.

CAPITOLO 34

Amministrare senza interfaccia grafica

Sui server, l'interfaccia grafica è spesso assente o sconsigliata: consuma risorse, amplia la superficie d'attacco e non si presta all'automazione. PowerShell è lo strumento con cui si fa **tutto** ciò che la GUI permetterebbe - e molto di più, perché è ripetibile.

Server con GUI, Server Core e oltre

Windows Server può essere installato con l'esperienza desktop completa oppure come **Server Core**: nessun desktop, solo una console. Server Core è più leggero e sicuro, ed è pensato proprio per essere gestito da riga di comando e da remoto. In questo scenario PowerShell non è opzionale: è l'interfaccia primaria.

NOTA —

La filosofia è la stessa dei sistemi industriali headless: un PLC o un gateway di campo non hanno un monitor: si configurano e si diagnosticano da un'altra postazione. Server Core porta questo approccio nel mondo dei server Windows.

SConfig e i primi passi su Server Core

All'avvio, Server Core offre **SConfig**, un menu testuale per le configurazioni iniziali (nome macchina, rete, dominio, aggiornamenti). Superata la configurazione di base, si prosegue con i cmdlet.

Esempio 34.1 - prime configurazioni di un server da PowerShell.

```
# Operazioni iniziali tipiche, da PowerShell:
Rename-Computer -NewName 'SRV-OT-01' -Restart           # rinomina e riavvia

# Informazioni essenziali sul sistema:
Get-ComputerInfo -Property OsName, OsVersion, CsName, OsArchitecture

# Fuso orario e data:
Set-TimeZone -Id 'W. Europe Standard Time'
Get-Date
```

Windows Admin Center: la GUI moderna basata su comandi

Per chi desidera comunque un'interfaccia, **Windows Admin Center** è una console web che amministra i server (anche Core) traducendo le azioni in operazioni PowerShell sottostanti. È il ponte tra la comodità visiva e la sostanza a riga di comando: utile, ma ciò che conta resta sotto, ed è bene saperlo padroneggiare direttamente.

Fonti

- Microsoft Learn, *Server Core e SConfig* - learn.microsoft.com/windows-server
- Microsoft Learn, *Windows Admin Center* - learn.microsoft.com/windows-server/manage/windows-admin-center

CAPITOLO 35

PowerShell Remoting: amministrare a distanza

Il **Remoting** e la capacita di eseguire comandi su computer remoti come se si fosse seduti davanti ad essi. E il cuore dell'amministrazione moderna: una sola postazione governa decine di server.

Abilitare il Remoting

Il Remoting classico su Windows si basa sul protocollo **WinRM**. Si abilita con un cmdlet (di norma gia attivo su Windows Server).

Esempio 35.1 - abilitare e verificare il Remoting (WinRM).

```
# Sul server da gestire (una volta sola, come amministratore):
Enable-PSRemoting -Force

# Verificare che il Remoting risponda:
Test-WSMan -ComputerName 'SRV-OT-01'
```

ATTENZIONE —

Abilitare il Remoting apre un canale di amministrazione potente: va fatto in una rete controllata, con autenticazione robusta e preferibilmente segmentando le reti. In ambito OT, l'accesso remoto ai sistemi va progettato con la stessa attenzione di un accesso fisico al quadro.

Eseguire comandi remoti: Invoke-Command

Invoke-Command esegue un blocco di codice su uno o piu computer e riporta indietro i risultati come **oggetti**. E lo strumento per operare su molte macchine in parallelo.

Esempio 35.2 - Invoke-Command su piu server contemporaneamente.

```
# Stesso comando su piu server, in parallelo, risultati come oggetti:
$server = 'SRV-OT-01', 'SRV-OT-02', 'SRV-SCADA-01'
Invoke-Command -ComputerName $server -ScriptBlock {
    [pscustomobject]@{
        Nome     = $env:COMPUTERNAME
        Uptime   = (Get-Date) - (Get-CimInstance Win32_OperatingSystem).LastBootUpTime
        DiscoC   = (Get-PSDrive C).Free / 1GB
    }
} | Format-Table -AutoSize
```

Sessioni interattive e persistenti

Enter-PSSession apre una sessione interattiva su un singolo server (come essere li); **New-PSSession** crea sessioni **persistenti** riutilizzabili per piu comandi, evitando di riconnettersi ogni volta.

Esempio 35.3 - Enter-PSSession e sessioni persistenti.

```
# Sessione interattiva su un server (il prompt cambia):
Enter-PSSession -ComputerName 'SRV-OT-01'
# [SRV-OT-01]: PS C:\> ... comandi ...
Exit-PSSession

# Sessioni persistenti, utili per piu operazioni sugli stessi host:
$sess = New-PSSession -ComputerName 'SRV-OT-01', 'SRV-OT-02'
Invoke-Command -Session $sess -ScriptBlock { Get-Service W3SVC }
Remove-PSSession -Session $sess
```

Remoting su SSH e multiplatforma

PowerShell 7 supporta il Remoting anche su **SSH**, oltre che su WinRM. Questo permette sessioni tra Windows, Linux e macOS con un trasporto standard e diffuso - utile in ambienti eterogenei, sempre più comuni anche nelle reti OT.

Esempio 35.4 - Remoting basato su SSH in PowerShell 7.

```
# Remoting via SSH (richiede OpenSSH configurato sull'host remoto):  
Invoke-Command -HostName 'server-linux' -UserName 'admin' -ScriptBlock {  
    Get-Process | Sort-Object CPU -Descending | Select-Object -First 5  
}
```

SUGGERIMENTO —

Invoke-Command su una lista di server è l'equivalente di una campagna di manutenzione coordinata su più impianti: una sola istruzione, applicata ovunque, con il riscontro di ciascun sito che torna in forma strutturata e confrontabile.

Fonti

- Microsoft Learn, *about_Remote*, *Invoke-Command*, *New-PSSession* - learn.microsoft.com/powershell
- Microsoft Learn, *PowerShell Remoting over SSH* - learn.microsoft.com/powershell/scripting/security/remoting/ssh-remoting

CAPITOLO 36

Ruoli e funzionalita del server

Aggiungere a un server il ruolo di web server, di file server o di controller di dominio - operazioni che nella GUI si fanno con *Server Manager* - in PowerShell sono un solo comando, e quindi automatizzabili su molte macchine.

Elencare, installare, rimuovere

Esempio 36.1 - gestione dei ruoli con i cmdlet WindowsFeature.

```
# Quali ruoli/funzionalita ci sono e quali sono installati:
Get-WindowsFeature | Where-Object Installed | Select-Object Name, DisplayName

# Installare un ruolo, includendo gli strumenti di gestione:
Install-WindowsFeature -Name Web-Server -IncludeManagementTools

# Rimuovere una funzionalita non necessaria:
Uninstall-WindowsFeature -Name Web-Server
```

NOTA —

Il parametro `-IncludeManagementTools` aggiunge anche gli strumenti di amministrazione del ruolo (cmdlet e console): dimenticarlo è un errore frequente che lascia il ruolo installato ma poco gestibile dalla macchina stessa.

Installazione coordinata su piu server

Unendo `Invoke-Command` e i cmdlet dei ruoli si configura un'intera flotta in modo uniforme.

Esempio 36.2 - applicare un ruolo a piu server insieme.

```
# Garantire lo stesso ruolo su piu server con un'unica istruzione:
Invoke-Command -ComputerName 'SRV-OT-01','SRV-OT-02' -ScriptBlock {
    Install-WindowsFeature -Name 'SNMP-Service' -IncludeManagementTools
}
```

Fonti

- Microsoft Learn, *Install-WindowsFeature*, *Get-WindowsFeature* - learn.microsoft.com/powershell

CAPITOLO 37

Gestione della rete

Indirizzi IP, DNS, gateway, regole del firewall: la configurazione di rete, spesso fatta a mano nella GUI, è completamente gestibile da PowerShell - aspetto cruciale per documentare e replicare la configurazione di rete dei server OT.

Indirizzi IP, gateway e DNS

Esempio 37.1 - configurare IP statico, gateway e DNS.

```
# Stato delle schede e degli indirizzi attuali:
Get-NetAdapter | Select-Object Name, Status, LinkSpeed
Get-NetIPAddress -AddressFamily IPv4 | Select-Object InterfaceAlias, IPAddress

# Assegnare un IP statico con gateway (interfaccia 'LAN-OT'):
New-NetIPAddress -InterfaceAlias 'LAN-OT' -IPAddress 192.168.10.5 `
  -PrefixLength 24 -DefaultGateway 192.168.10.1

# Impostare i server DNS:
Set-DnsClientServerAddress -InterfaceAlias 'LAN-OT' `
  -ServerAddresses 192.168.10.2, 192.168.10.3
```

Diagnostica di rete

Test-NetConnection è il coltellino svizzero della diagnostica: verifica raggiungibilità, porta TCP aperta, percorso. Sostituisce e supera il vecchio **ping**.

Esempio 37.2 - Test-NetConnection per la diagnostica.

```
# E' raggiungibile? La porta Modbus 502 è aperta sul dispositivo?
Test-NetConnection -ComputerName 192.168.10.20 -Port 502

# Diagnostica estesa (instradamento, latenza):
Test-NetConnection -ComputerName 'gateway-impianto' -TraceRoute
```

Il firewall di Windows

Esempio 37.3 - gestione del firewall con i cmdlet NetSecurity.

```
# Elencare le regole abilitate in ingresso:
Get-NetFirewallRule -Direction Inbound -Enabled True |
  Select-Object DisplayName, Action | Sort-Object DisplayName

# Aprire una porta specifica (es. servizio applicativo su 8080):
New-NetFirewallRule -DisplayName 'App Impianto 8080' -Direction Inbound `
  -Protocol TCP -LocalPort 8080 -Action Allow
```

ATTENZIONE —

Modificare il firewall di un server in produzione richiede prudenza: una regola sbagliata può isolare la macchina o, peggio, esporla. In ambito OT ogni apertura va giustificata e documentata, secondo il principio della minima esposizione.

SUGGERIMENTO —

Configurare la rete via script significa poterla **versionare**: la configurazione di rete di un server smette di essere una serie di clic difficili da ricostruire e diventa un documento eseguibile, replicabile identico su un secondo server di riserva.

Fonti

- Microsoft Learn, *NetTCP/IP (New-NetIPAddress)*, *DnsClient*, *NetSecurity* - learn.microsoft.com/powershell
- Microsoft Learn, *Test-NetConnection* - learn.microsoft.com/powershell

CAPITOLO 38

Active Directory

Nei domini Windows, **Active Directory** (AD) gestisce utenti, gruppi e computer. Le operazioni che nella GUI si fanno con *Utenti e computer di Active Directory* si automatizzano con il modulo **ActiveDirectory** - indispensabile quando gli account da creare o modificare sono molti.

Consultare utenti, gruppi, computer

Esempio 38.1 - interrogare Active Directory.

```
Import-Module ActiveDirectory

# Cercare utenti (con filtro) e mostrarne alcune proprietà:
Get-ADUser -Filter "Department -eq 'Automazione'" `
    -Properties Department, Title | Select-Object Name, Title

# Account computer registrati nel dominio:
Get-ADComputer -Filter * | Select-Object Name, OperatingSystem

# Membri di un gruppo:
Get-ADGroupMember -Identity 'Operatori-SCADA' | Select-Object Name
```

Creare account e gestire l'appartenenza

Esempio 38.2 - creare un utente e assegnarlo a un gruppo.

```
# Creare un nuovo utente con password iniziale sicura:
$pwd = Read-Host 'Password iniziale' -AsSecureString
New-ADUser -Name 'Mario Rossi' -SamAccountName 'mrossi' `
    -UserPrincipalName 'mrossi@azienda.local' `
    -Path 'OU=Automazione,DC=azienda,DC=local' `
    -AccountPassword $pwd -Enabled $true -ChangePasswordAtLogon $true

# Aggiungerlo a un gruppo:
Add-ADGroupMember -Identity 'Operatori-SCADA' -Members 'mrossi'
```

Creazione di massa da un file CSV

Questo è lo scenario in cui PowerShell brilla davvero: creare decine di account leggendoli da un CSV, in pochi secondi e senza errori manuali.

Esempio 38.3 - creazione massiva di utenti da CSV.

```
# operatori.csv con colonne: Nome,Sam,OU
$pwdDefault = ConvertTo-SecureString 'Tempor@neal!' -AsPlainText -Force

Import-Csv 'C:\hr\operatori.csv' | ForEach-Object {
    New-ADUser -Name $_.Nome -SamAccountName $_.Sam `
        -Path $_.OU -AccountPassword $pwdDefault `
        -Enabled $true -ChangePasswordAtLogon $true
    Write-Host "Creato: $(($_.Sam))"
}
```

ATTENZIONE —

Le operazioni su Active Directory hanno effetti immediati e diffusi su tutto il dominio. Provare sempre i filtri con **Get-ADUser** prima di lanciare modifiche, e considerare **-WhatIf** dove disponibile. Un errore di massa qui è tanto rapido quanto difficile da annullare.

Fonti

- Microsoft Learn, *ActiveDirectory* module (*Get-ADUser*, *New-ADUser*, *Add-ADGroupMember*) - learn.microsoft.com/powershell

CAPITOLO 39

Storage, condivisioni e permessi

Dischi, volumi, cartelle condivise e permessi: la gestione dello storage e dei file server, anch'essa interamente scriptabile.

Dischi e volumi

Esempio 39.1 - ispezionare dischi e volumi.

```
# Panoramica di dischi, partizioni e volumi:
Get-Disk | Select-Object Number, FriendlyName, @{N='GB';E=[math]::Round($_.Size/1GB)}
Get-Volume | Select-Object DriveLetter, FileSystemLabel,
    @{N='LiberiGB';E=[math]::Round($_.SizeRemaining/1GB,1)}
```

Condivisioni SMB

Esempio 39.2 - gestione delle condivisioni SMB.

```
# Creare una condivisione e concedere l'accesso a un gruppo:
New-SmbShare -Name 'Ricette' -Path 'D:\produzione\ricette' `
    -FullAccess 'AZIENDA\Operatori-SCADA'

# Elencare le condivisioni e chi vi ha accesso:
Get-SmbShare
Get-SmbShareAccess -Name 'Ricette'
```

Permessi NTFS: leggere e modificare le ACL

I permessi del file system (ACL) si leggono con **Get-Acl** e si applicano con **Set-Acl**. La logica è prima costruire una regola di accesso, poi aggiungerla alla lista esistente.

Esempio 39.3 - leggere e impostare i permessi NTFS.

```
# Leggere i permessi di una cartella:
(Get-Acl 'D:\produzione\ricette').Access |
    Select-Object IdentityReference, FileSystemRights, AccessControlType

# Concedere 'Modifica' a un gruppo, preservando i permessi esistenti:
$acl = Get-Acl 'D:\produzione\ricette'
$regola = New-Object System.Security.AccessControl.FileSystemAccessRule(
    'AZIENDA\Operatori-SCADA', 'Modify', 'ContainerInherit,ObjectInherit', 'None', 'Allow')
$acl.AddAccessRule($regola)
Set-Acl -Path 'D:\produzione\ricette' -AclObject $acl
```

NOTA —

Le ACL sono delicate: un **Set-Acl** che sostituisce invece di aggiungere può togliere l'accesso a tutti. Per questo si parte **sempre** da **Get-Acl**, si aggiunge la regola e si riapplica, verificando prima su una cartella di prova.

Fonti

- Microsoft Learn, *Storage* e *SmbShare* modules - learn.microsoft.com/powershell
- Microsoft Learn, *Get-Acl*, *Set-Acl* - learn.microsoft.com/powershell

CAPITOLO 40

Servizi, processi ed eventi

Il controllo quotidiano di un server: stato dei servizi, processi in esecuzione, registro eventi. Sono le verifiche che si farebbero con *Servizi*, *Gestione attività* e *Visualizzatore eventi* - qui tutte a comando e automatizzabili.

Servizi

Esempio 40.1 - monitorare e gestire i servizi.

```
# Stato dei servizi critici per l'impianto:
Get-Service -Name 'W3SVC','MSSQLSERVER','OPCUA*' -ErrorAction SilentlyContinue |
    Select-Object Name, Status, StartType

# Riavviare un servizio e impostarne l'avvio automatico:
Restart-Service -Name 'W3SVC'
Set-Service -Name 'W3SVC' -StartupType Automatic
```

Processi

Esempio 40.2 - ispezionare e terminare processi.

```
# I dieci processi che consumano piu memoria:
Get-Process | Sort-Object WorkingSet -Descending |
    Select-Object -First 10 Name, Id,
        @{N='MB';E={[math]::Round($_.WorkingSet/1MB)}}

# Terminare un processo bloccato (con prudenza):
Get-Process -Name 'app_bloccata' -ErrorAction SilentlyContinue |
    Stop-Process -WhatIf
```

Il registro eventi con Get-WinEvent

Get-WinEvent interroga i log eventi di Windows in modo efficiente. E lo strumento per le indagini: errori di sistema, accessi, comportamenti anomali.

Esempio 40.3 - filtrare gli eventi recenti con Get-WinEvent.

```
# Ultimi 20 errori e avvisi del log di sistema delle ultime 24 ore:
Get-WinEvent -FilterHashtable @{
    LogName    = 'System'
    Level      = 1,2,3          # 1=Critico 2=Errore 3=Avviso
    StartTime  = (Get-Date).AddHours(-24)
} -MaxEvents 20 |
    Select-Object TimeCreated, Id, LevelDisplayName, ProviderName, Message
```

SUGGERIMENTO —

Per un tecnico abituato a leggere i log allarmi di uno SCADA, **Get-WinEvent** con un *filter hashtable* è familiare: si filtra per tempo, gravità e sorgente, e si ottiene l'elenco strutturato degli eventi - solo che qui la 'macchina' è il server Windows.

Fonti

- Microsoft Learn, *Get-Service*, *Get-Process*, *Get-WinEvent* - learn.microsoft.com/powershell

CAPITOLO 41

IIS, Hyper-V e configurazione dichiarativa

Chiudiamo la panoramica con tre ambiti avanzati ma frequenti: il web server IIS, la virtualizzazione con Hyper-V e la configurazione **dichiarativa** con DSC. L'obiettivo non è l'eshaustività, ma mostrare che anche questi mondi parlano PowerShell.

IIS: il web server

Esempio 41.1 - gestione di base di IIS via PowerShell.

```
Import-Module WebAdministration

# Elencare i siti e crearne uno nuovo:
Get-Website | Select-Object Name, State, PhysicalPath
New-Website -Name 'Portale-OT' -Port 8081 `
    -PhysicalPath 'D:\siti\portale-ot'

# Riavviare un pool di applicazioni:
Restart-WebAppPool -Name 'Portale-OT'
```

Hyper-V: le macchine virtuali

Esempio 41.2 - gestire macchine virtuali con i cmdlet Hyper-V.

```
# Stato delle VM e avvio/arresto:
Get-VM | Select-Object Name, State,
    @{N='RAM_GB';E={$_.MemoryAssigned/1GB}}
Start-VM -Name 'VM-Test-SCADA'
Stop-VM -Name 'VM-Test-SCADA' -Save          # salva lo stato

# Creare un checkpoint (snapshot) prima di una modifica rischiosa:
Checkpoint-VM -Name 'VM-Test-SCADA' -SnapshotName 'pre-aggiornamento'
```

SUGGERIMENTO —

Il checkpoint di una VM prima di un intervento è la rete di sicurezza per eccellenza: se qualcosa va storto si torna allo stato precedente in un istante. È il 'punto di ripristino' che si vorrebbe sempre avere anche prima di toccare un sistema in produzione.

Desired State Configuration: descrivere lo stato voluto

DSC rovescia la prospettiva: invece di elencare i comandi da eseguire, si **descrive lo stato desiderato** del sistema (quali funzionalità, quali servizi attivi, quali file presenti) e il motore si occupa di portarlo e mantenerlo in quello stato. È un approccio **dichiarativo**, vicino nello spirito alla programmazione a stati di un automatismo.

Esempio 41.3 - una configurazione DSC dichiarativa.

```
Configuration ServerWeb0T {
    Node 'SRV-OT-01' {
        WindowsFeature IIS {
            Name     = 'Web-Server'
            Ensure   = 'Present'      # deve essere installato
        }
        Service W3SVC {
            Name     = 'W3SVC'
            State    = 'Running'     # deve essere in esecuzione
        }
    }
}

# Si genera la configurazione e la si applica:
# ServerWeb0T ; Start-DscConfiguration -Path .\ServerWeb0T -Wait -Verbose
```

NOTA —

L'idea dichiarativa - *dico come deve essere, non passo per passo cosa fare* - è potente per garantire che molti server restino **conformi** a uno standard nel tempo, correggendo da soli le derive di configurazione. È un principio che continua a evolversi negli strumenti più recenti di gestione della configurazione.

Fonti

- Microsoft Learn, *WebAdministration* e *Hyper-V* modules - learn.microsoft.com/powershell
- Microsoft Learn, *Desired State Configuration (DSC)* - learn.microsoft.com/powershell/dsc

PARTE VI

PowerShell nell'automazione industriale

Mettere PowerShell al servizio dell'impianto: chiamare API, dialogare via TCP e seriale con i dispositivi, raccogliere misure, salvarle su database, produrre report e inviare allarmi. Qui la shell incontra il campo.

CAPITOLO 42

Chiamare API REST e servizi web

Sempre piu dispositivi e sistemi industriali - gateway, storici, piattaforme IIoT, controllori moderni - espongono **API REST**: si interrogano via HTTP scambiando JSON. PowerShell le consuma nativamente con `Invoke-RestMethod`, senza librerie aggiuntive.

Invoke-RestMethod: leggere e scrivere dati

`Invoke-RestMethod` esegue la chiamata HTTP e converte **automaticamente** la risposta JSON in oggetti PowerShell, pronti per la pipeline. E la differenza tra ricevere un blob di testo e ricevere dati strutturati.

Esempio 42.1 - GET e POST verso un'API REST con conversione JSON automatica.

```
# GET: leggere lo stato da un gateway che espone API REST
$base = 'http://192.168.10.50/api'
$stato = Invoke-RestMethod -Uri "$base/v1/status" -Method Get
$stato.pompe | Select-Object id, portata, stato

# POST: inviare un set-point in formato JSON
$corpo = @{ pompa = 1; setpoint = 42.5 } | ConvertTo-Json
Invoke-RestMethod -Uri "$base/v1/setpoint" -Method Post `
  -Body $corpo -ContentType 'application/json'
```

Autenticazione e intestazioni

Le API protette richiedono di solito un *token* o credenziali, passati nelle intestazioni HTTP. Il token va gestito con i metodi sicuri della Parte III, mai scritto in chiaro nello script.

Esempio 42.2 - chiamata autenticata con token Bearer nelle intestazioni.

```
# Recupero del token dal vault (vedi Parte III), poi chiamata autenticata
$token = Get-Secret -Name 'GatewayOT/Token' -AsPlainText
$intest = @{ Authorization = "Bearer $token" }

Invoke-RestMethod -Uri "$base/v1/misure" -Headers $intest -Method Get |
  Select-Object timestamp, tag, valore
```

Invoke-WebRequest quando serve la risposta grezza

Se occorre il controllo completo sulla risposta (codice di stato, intestazioni, contenuto non-JSON) si usa `Invoke-WebRequest`, che restituisce l'intero oggetto risposta invece dei soli dati.

Esempio 42.3 - Invoke-WebRequest per ispezionare la risposta HTTP.

```
$r = Invoke-WebRequest -Uri "$base/v1/health" -Method Get
$r.StatusCode # 200 se tutto ok
$r.Headers['Content-Type']
if ($r.StatusCode -ne 200) { Write-Warning 'Gateway non sano' }
```

ATTENZIONE —

Verso dispositivi in HTTPS con certificati interni puo servire gestire la validazione del certificato. Non disabilitare i controlli alla leggera: in OT un canale non verificato e un rischio. Meglio installare correttamente la catena di certificati interna.

SUGGERIMENTO —

`Invoke-RestMethod` rende un'API REST un *fornitore di oggetti*: una volta che le misure tornano come oggetti, valgono tutti gli strumenti già visti - `Where-Object`, `Export-Csv`, calcoli - senza dover analizzare manualmente il testo.

Fonti

- Microsoft Learn, *Invoke-RestMethod*, *Invoke-WebRequest* - learn.microsoft.com/powershell

CAPITOLO 43

Comunicazione TCP e protocolli industriali

Molti dispositivi di campo non parlano HTTP, ma protocolli su TCP come **Modbus TCP**, **OPC UA** o **MQTT**. PowerShell, grazie all'accesso diretto a .NET, può aprire socket TCP e dialogare a basso livello: utile per diagnostica, test e raccolte dati semplici.

Un client TCP grezzo con .NET

La classe .NET `System.Net.Sockets.TcpClient` permette di aprire una connessione e scambiare byte. È il mattone su cui poggiano i protocolli industriali su TCP.

Esempio 43.1 - una funzione generica per scambiare byte su TCP via .NET.

```
function Send-TcpBytes {  
    param([string]$IP, [int]$Porta, [byte[]]$Dati, [int]$TimeoutMs = 2000)  
    $client = [System.Net.Sockets.TcpClient]::new()  
    try {  
        $client.Connect($IP, $Porta)  
        $stream = $client.GetStream()  
        $stream.ReadTimeout = $TimeoutMs  
        $stream.Write($Dati, 0, $Dati.Length)  
        $buf = New-Object byte[] 256  
        $n = $stream.Read($buf, 0, $buf.Length)  
        return $buf[0..($n - 1)]  
    }  
    finally { $client.Close() } # sempre chiudere la connessione  
}
```

Esempio: lettura Modbus TCP di registri holding

Il protocollo **Modbus TCP** incapsula la richiesta in una intestazione MBAP seguita dal codice funzione. Per leggere registri *holding* (funzione `0x03`) si compone una richiesta di pochi byte. Vediamo come costruirla e inviarla con la funzione precedente.

Esempio 43.2 - comporre e interpretare una lettura Modbus TCP grezza.

```
# Richiesta Modbus TCP: leggi 2 registri holding dall'indirizzo 0, unit 1  
# MBAP: TID=0001 PID=0000 LEN=0006 UID=01 | PDU: FC=03 ADDR=0000 QTY=0002  
$richiesta = [byte[]](0x00,0x01, 0x00,0x00, 0x00,0x06, 0x01,  
                      0x03, 0x00,0x00, 0x00,0x02)  
  
$risposta = Send-TcpBytes -IP '192.168.10.20' -Porta 502 -Dati $richiesta  
  
# I valori dei registri iniziano al 10mo byte (2 byte ciascuno, big-endian)  
$reg1 = [int]$risposta[9] * 256 + $risposta[10]  
$reg2 = [int]$risposta[11] * 256 + $risposta[12]  
[pscustomobject]@{ Registro0 = $reg1; Registro1 = $reg2 }
```

NOTA —

Per un uso intensivo di Modbus conviene un modulo dedicato dalla Gallery o una libreria .NET specializzata, che gestiscono CRC, endianness, tipi e riconessioni. L'esempio grezzo serve a capire il protocollo e per test diagnostici rapidi, non come driver di produzione.

OPC UA e MQTT

- **OPC UA** - lo standard di interoperabilità industriale. Da PowerShell si usa tramite librerie .NET dedicate (lo stack OPC UA) o moduli della community: si crea una sessione verso il server, si leggono/scrivono i nodi.

- **MQTT** - il protocollo leggero publish/subscribe diffuso nell'IIoT. Anche qui si ricorre a una libreria .NET (ad esempio uno dei client MQTT per .NET) o a un modulo: ci si sottoscrive a un *topic* e si reagisce ai messaggi.

SUGGERIMENTO —

La regola pratica: per **diagnostica e test** un socket TCP grezzo e perfetto; per **integrazioni stabili** si appoggia PowerShell a una libreria .NET o a un modulo del protocollo, che gestisce i dettagli ostici. PowerShell resta la 'colla' che orchestra, registra e notifica.

Fonti

- Microsoft Learn, *System.Net.Sockets.TcpClient* (.NET API) - learn.microsoft.com/dotnet/api
- Specifiche pubbliche Modbus (Modbus Organization) e OPC UA (OPC Foundation).

CAPITOLO 44

Porte seriali e strumenti di campo

Molti strumenti - analizzatori, bilance, lettori, dispositivi Modbus RTU - comunicano ancora via **porta seriale** (RS-232/RS-485). PowerShell vi accede tramite la classe .NET `System.IO.Ports.SerialPort`.

Aprire una porta e leggere

Esempio 44.1 - leggere da una porta seriale con `SerialPort` .NET.

```
# Elencare le porte seriali disponibili sul sistema:
[System.IO.Ports.SerialPort]::GetPortNames()

# Aprire COM3 a 9600 8N1 e leggere una riga inviata dallo strumento:
$porta = [System.IO.Ports.SerialPort]::new('COM3', 9600, 'None', 8, 'One')
$porta.ReadTimeout = 2000
try {
    $porta.Open()
    $riga = $porta.ReadLine()      # es. una misura terminata da newline
    Write-Host "Letto dallo strumento: $riga"
}
catch [TimeoutException] { Write-Warning 'Nessun dato entro il timeout' }
finally { if ($porta.IsOpen) { $porta.Close() } }
```

Scrivere un comando e attendere la risposta

Molti strumenti seguono uno schema *richiesta/risposta*: si invia un comando testuale e si legge la replica. La gestione dei timeout è essenziale, perché un dispositivo che non risponde non deve bloccare lo script all'infinito.

Esempio 44.2 - schema richiesta/risposta su seriale con timeout.

```
$porta = [System.IO.Ports.SerialPort]::new('COM3', 9600)
$porta.ReadTimeout = 1500; $porta.NewLine = "`r`n"
try {
    $porta.Open()
    $porta.WriteLine('READ?')      # comando allo strumento
    Start-Sleep -Milliseconds 200
    $risposta = $porta.ReadLine()
    [pscustomobject]@{ Ora = Get-Date; Valore = $risposta.Trim() }
}
finally { if ($porta.IsOpen) { $porta.Close() } }
```

ATTENZIONE —

Chiudere **sempre** la porta nel blocco **finally**: una porta seriale lasciata aperta resta bloccata e nessun altro processo potrà usarla finché lo script (o la sessione) non termina. È l'equivalente di un morsetto lasciato sotto tensione: va rilasciato in modo pulito.

SUGGERIMENTO —

Per chi conosce Modbus RTU: l'incapsulamento è diverso dal TCP (niente MBAP, c'è il CRC finale), ma il principio in PowerShell è lo stesso - comporre i byte, scriverli su `SerialPort`, leggere e validare la risposta. Per l'uso reale, una libreria che gestisca il CRC e i tempi di silenzio inter-frame è fortemente consigliata.

Fonti

- Microsoft Learn, `System.IO.Ports.SerialPort` (.NET API) - learn.microsoft.com/dotnet/api

CAPITOLO 45

Database e persistenza dei dati

Raccogliere misure non basta: vanno **conservate** per storici, analisi e tracciabilità. PowerShell sa scrivere e leggere database relazionali, dal SQL Server aziendale a soluzioni leggere come SQLite.

SQL Server con il modulo SqlServer

Il modulo **SqlServer** offre **Invoke-Sqlcmd**, che esegue query e restituisce le righe come oggetti. E il modo più diretto per interrogare o alimentare un database SQL Server.

Esempio 45.1 - interrogare SQL Server con Invoke-Sqlcmd.

```
Install-Module SqlServer -Scope CurrentUser # una volta sola

# Leggere le ultime misure da una tabella storica:
$q = 'SELECT TOP 10 Tag, Valore, Istante FROM Misure ORDER BY Istante DESC'
Invoke-Sqlcmd -ServerInstance 'SRV-SQL-01' -Database 'StoricoOT' -Query $q |
    Format-Table -AutoSize
```

Scrivere misure senza dipendenze, con .NET

Se non si vuole installare moduli, si usa direttamente il client SQL di .NET. Importante: per inserire dati usare **query parametrizzate**, che evitano errori di formattazione e, soprattutto, le iniezioni SQL.

Esempio 45.2 - inserire dati con query parametrizzata (System.Data.SqlClient).

```
function Add-Misura {
    param([string]$Tag, [double]$Valore)
    $cs = 'Server=SRV-SQL-01;Database=StoricoOT;Integrated Security=true;'
    $conn = [System.Data.SqlClient.SqlConnection]::new($cs)
    try {
        $conn.Open()
        $cmd = $conn.CreateCommand()
        $cmd.CommandText =
            'INSERT INTO Misure (Tag, Valore, Istante) VALUES (@t, @v, SYSDATETIME())'
        [void]$cmd.Parameters.AddWithValue('@t', $Tag) # parametri = niente injection
        [void]$cmd.Parameters.AddWithValue('@v', $Valore)
        [void]$cmd.ExecuteNonQuery()
    }
    finally { $conn.Close() }
}
Add-Misura -Tag 'LIV_VASCA_1' -Valore 78.4
```

ATTENZIONE —

Non costruire **mai** le query concatenando valori nel testo ("**... VALUES ('\$tag')**"): è la porta aperta alle iniezioni SQL e agli errori con apici e tipi. I parametri (**@t**, **@v**) sono la pratica corretta, sempre.

NOTA —

Per storici leggeri e locali, **SQLite** è un'ottima scelta (un singolo file, nessun server). Si usa tramite un modulo dedicato dalla Gallery o la libreria .NET di SQLite: utile su una postazione di campo che deve bufferizzare dati prima di inviarli al sistema centrale.

Fonti

- Microsoft Learn, *SqlServer module (Invoke-Sqlcmd)* - learn.microsoft.com/powershell
- Microsoft Learn, *System.Data.SqlClient (.NET API)* - learn.microsoft.com/dotnet/api

CAPITOLO 46

Reportistica: CSV, Excel e PDF

Il dato raccolto diventa utile quando è **leggibile** da chi decide: un report giornaliero, un foglio di calcolo, un documento. PowerShell produce tutti questi formati, spesso in poche righe.

CSV: il report universale

Esempio 46.1 - generare un report CSV datato.

```
# Report giornaliero degli allarmi, pronto per Excel/gestionale:
Get-WinEvent -FilterHashtable @{ LogName='System'; Level=1,2;
  StartTime=(Get-Date).Date } |
  Select-Object TimeCreated, Id, ProviderName, LevelDisplayName |
  Export-Csv -Path "D:\report\allarmi_$(Get-Date -f yyyyMMdd).csv" `
    -NoTypeInformation -Encoding UTF8
```

Excel nativo con il modulo ImportExcel

Il modulo **ImportExcel** crea veri file **.xlsx** (con formattazione, tabelle, grafici) **senza** che Excel sia installato. È lo strumento ideale per report professionali destinati agli uffici.

Esempio 46.2 - report Excel formattato con ImportExcel.

```
Install-Module ImportExcel -Scope CurrentUser

# Esportare misure in un foglio Excel formattato, con tabella e grafico:
$dati = Invoke-Sqlcmd -ServerInstance 'SRV-SQL-01' -Database 'StoricoOT' `
  -Query 'SELECT Tag, Valore, Istante FROM Misure WHERE Istante > DATEADD(day,-1,SYSDATETIME())'

$dati | Export-Excel -Path 'D:\report\giornaliero.xlsx' `
  -WorksheetName 'Misure' -TableName 'Misure' -AutoSize `
  -Title 'Report misure ultime 24h' -IncludePivotChart `
  -PivotRows Tag -PivotData @{ Valore = 'Average' }
```

PDF: la via pratica

PowerShell non genera PDF nativamente. Le strade collaudate sono due: produrre **HTML** con **ConvertTo-Html** e poi convertirlo in PDF con uno strumento esterno, oppure usare una libreria .NET di generazione PDF. Per molti casi, un HTML ben fatto è già un report condivisibile.

Esempio 46.3 - report HTML con ConvertTo-Html (base per PDF/e-mail).

```
# Report HTML con stile, facilmente convertibile in PDF o inviabile via mail:
$stile = '<style>body{font-family:Segoe UI}table{border-collapse:collapse}
th,td{border:1px solid #ccc;padding:6px}</style>'

Get-Service | Where-Object Status -eq 'Running' |
  Select-Object Name, Status, StartType |
  ConvertTo-Html -Title 'Servizi attivi' -Head $stile `
    -PreContent '<h2>Servizi in esecuzione</h2>' |
  Set-Content -Path 'D:\report\servizi.html' -Encoding UTF8
```

SUGGERIMENTO —

Schema vincente per un report periodico: una **query** estrae i dati, **Export-Excel** o **ConvertTo-Html** li impagina, un'**attività pianificata** (Parte IV) lo esegue ogni mattina, e una **notifica** (prossimo capitolo) lo recapita. Tre mattoni già visti, combinati in un servizio completo e autonomo.

Fonti

- Microsoft Learn, *Export-Csv, ConvertTo-Html* - learn.microsoft.com/powershell
- ImportExcel - github.com/dfinke/ImportExcel ; PowerShell Gallery

CAPITOLO 47

Allarmi e notifiche

Un sistema di monitoraggio serve a poco se non **avvisa** quando qualcosa non va. PowerShell può recapitare notifiche via e-mail o verso le piattaforme di messaggistica tramite *webhook*, chiudendo il ciclo: rileva, valuta, avvisa.

Notifiche via e-mail

Storicamente si usava `Send-MailMessage`. Microsoft lo considera oggi **obsoleto** (non garantisce i protocolli sicuri più recenti): resta utile su reti interne e server SMTP legacy, ma per nuovi sviluppi è preferibile una libreria .NET moderna (ad esempio un client come MailKit) o l'invio tramite API del provider di posta.

Esempio 47.1 - invio e-mail con Send-MailMessage (da considerare legacy).

```
# Approccio classico (obsoleto, ma diffuso su SMTP interni):
Send-MailMessage -SmtpServer 'smtp.azienda.local' -Port 25 `
  -From 'monitor-ot@azienda.local' -To 'manutenzione@azienda.local' `
  -Subject 'ALLARME livello vasca 1' `
  -Body 'Livello oltre soglia: 92%. Verificare la pompa di rilancio.'
# Nota: per nuovi progetti valutare MailKit o le API del provider.
```

Notifiche via webhook (Teams, Telegram, ...)

Le piattaforme moderne accettano notifiche tramite **webhook**: un URL a cui inviare un POST con un messaggio JSON. Si fa con `Invoke-RestMethod`, lo stesso cmdlet del capitolo sulle API.

Esempio 47.2 - notifica verso un webhook con Invoke-RestMethod.

```
# Esempio generico: inviare un avviso a un webhook (URL dal vault)
$webhook = Get-Secret -Name 'Notifiche/WebhookOT' -AsPlainText
$msg = @{ text = 'ALLARME: livello vasca 1 al 92% - verificare pompa' } |
  ConvertTo-Json
Invoke-RestMethod -Uri $webhook -Method Post -Body $msg `
  -ContentType 'application/json'
```

Un monitor completo, in pochi mattoni

Mettiamo insieme tutto: leggere un valore (via API o Modbus), confrontarlo con una soglia, e notificare se superata - il nucleo di qualunque sorveglianza automatica.

Esempio 47.3 - sorveglianza con soglia e notifica, da pianificare.

```
function Watch-Livello {
    param([double]$Soglia = 90)
    $stato = Invoke-RestMethod -Uri 'http://192.168.10.50/api/v1/status'
    $liv = [double]$stato.vasca1.livello
    Write-Log "Livello vasca1: $liv%" # log (Parte IV)
    if ($liv -gt $Soglia) {
        $msg = @{ text = "ALLARME vasca1: $liv% (soglia $Soglia%)" } | ConvertTo-Json
        Invoke-RestMethod -Uri (Get-Secret -Name 'Notifiche/WebhookOT' -AsPlainText) `
          -Method Post -Body $msg -ContentType 'application/json'
    }
}
# Pianificato ogni 5 minuti (Parte IV), diventa una sentinella autonoma.
Watch-Livello -Soglia 88
```

SUGGERIMENTO —

Questo piccolo monitor riassume l'intera guida: variabili e confronti (Parte II), gestione sicura dei segreti (Parte III), funzioni e pianificazione (Parte IV), API e notifiche (Parte VI). PowerShell diventa il sistema nervoso che collega misure, decisioni e avvisi - un alleato concreto accanto a PLC e SCADA.

ATTENZIONE —

Una notifica automatica non sostituisce i sistemi di sicurezza certificati dell'impianto: PowerShell è un ottimo strumento di **supervisione e supporto**, non una funzione di sicurezza funzionale. Le protezioni critiche restano in capo ai dispositivi e alle logiche qualificate per quel compito.

Fonti

- Microsoft Learn, *Send-MailMessage* (segnalato come obsoleto) - learn.microsoft.com/powershell
- Microsoft Learn, *Invoke-RestMethod* - learn.microsoft.com/powershell

PARTE VII

Confronti tra shell

PowerShell, il vecchio cmd di MS-DOS e le shell di Linux a confronto: equivalenze dei comandi, differenze di filosofia, punti di forza e criteri di scelta. Per orientarsi e per migrare con consapevolezza.

CAPITOLO 48

Tre mondi a confronto

Chi amministra sistemi misti incontra prima o poi tre famiglie di shell: il **cmd** ereditato da MS-DOS, le shell **Unix/Linux** (su tutte *bash*) e **PowerShell**. Hanno storie e filosofie diverse, e conoscerne le equivalenze fa risparmiare tempo e errori.

La differenza di fondo: testo contro oggetti

La distinzione più importante l'abbiamo già incontrata nella Parte I, ma qui diventa il criterio guida. **cmd e bash trattano flussi di testo**: ogni comando produce righe di caratteri che il comando successivo deve analizzare. **PowerShell tratta oggetti**: lungo la pipeline scorrono strutture con proprietà tipizzate, non testo da interpretare.

Questa differenza spiega molte delle equivalenze che seguono: dove in *bash* si concatenano filtri di testo (*grep*, *awk*, *cut*, *sed*), in PowerShell si accede direttamente a una proprietà dell'oggetto. Le shell Unix compensano la natura testuale con strumenti di manipolazione del testo straordinariamente potenti e maturi; PowerShell evita a monte il problema lavorando su dati strutturati.

NOTA —

Nessun approccio è 'migliore' in assoluto. La filosofia Unix - tanti piccoli strumenti che fanno una cosa sola e si combinano via testo - è potentissima ed è alla base di mezzo mondo informatico. L'approccio a oggetti di PowerShell brilla quando i dati sono strutturati e quando si integra con l'ecosistema Windows e .NET.

Come leggere le tabelle di equivalenza

Nelle pagine seguenti, per ciascun compito si trovano il comando PowerShell, l'equivalente in *cmd* (ove esista) e quello tipico in Linux/*bash*. Attenzione: molte equivalenze sono **approssimative**. In particolare gli alias di PowerShell (*ls*, *cp*, *cat*, ...) *assomigliano* ai comandi Unix ma chiamano cmdlet che restituiscono oggetti, con opzioni diverse. Sono comodi alla tastiera, ma negli script si usano sempre i nomi completi dei cmdlet.

ATTENZIONE —

Gli alias 'unix-like' di PowerShell esistono solo su Windows e possono trarre in inganno: *ls* in PowerShell non è *ls* di Linux. Su PowerShell multiplatforma (Linux/macOS) alcuni alias che andrebbero in conflitto con i comandi nativi non sono nemmeno definiti. Negli script: nomi completi, mai alias.

CAPITOLO 49

Equivalenze: file e cartelle

Le operazioni più comuni - elencare, copiare, spostare, eliminare file e cartelle - in tutte e tre le shell. E il punto di partenza per chi migra da cmd o da bash.

Tabella 49.1 - file e cartelle: equivalenze tra le shell.

Compito	PowerShell (cmdlet)	cmd (DOS)	Linux (bash)
Elenca contenuto cartella	Get-ChildItem	dir	ls -l
Cambia cartella	Set-Location	cd	cd
Cartella corrente	Get-Location	cd	pwd
Crea cartella	New-Item -ItemType Directory	mkdir	mkdir
Copia file	Copy-Item	copy	cp
Copia ricorsiva cartelle	Copy-Item -Recurse	xcopy /E	cp -r
Sposta/rinomina	Move-Item	move / ren	mv
Elimina file	Remove-Item	del	rm
Elimina cartella ricorsiva	Remove-Item -Recurse	rmdir /S	rm -r
Mostra contenuto file	Get-Content	type	cat
Scrivi/crea file di testo	Set-Content	echo > file	echo > file
Accoda a un file	Add-Content	echo >> file	echo >> file
File esiste?	Test-Path	if exist	test -e / [-e]
Percorso assoluto	Resolve-Path	(n.d.)	realpath
Unisci parti di percorso	Join-Path	(manuale)	(manuale)

SUGGERIMENTO —

Per chi viene da DOS: `Get-ChildItem` ha gli alias `dir`, `ls` e `gci`. Restituisce però oggetti `FileInfo/DirectoryInfo`, quindi `(Get-ChildItem).Length`, `.LastWriteTime`, `.FullName` sono immediatamente disponibili senza analizzare testo.

CAPITOLO 50

Equivalenze: testo, ricerca e dati

Qui la differenza di filosofia si fa più evidente. In bash si filtra e si trasforma testo con strumenti dedicati; in PowerShell spesso si lavora sugli oggetti, ricorrendo al testo solo quando serve davvero.

Tabella 50.1 - testo, ricerca e manipolazione dei dati.

Compito	PowerShell	cmd (DOS)	Linux (bash)
Cerca testo in file	Select-String	find / findstr	grep
Filtra per condizione	Where-Object	(findstr)	grep / awk
Conta righe/elementi	Measure-Object	find /C	wc -l
Ordina	Sort-Object	sort	sort
Elementi unici	Select-Object -Unique	(n.d.)	uniq
Prime N righe	Select-Object -First N	(n.d.)	head -n N
Ultime N righe	Select-Object -Last N	(n.d.)	tail -n N
Segui un file in tempo reale	Get-Content -Wait	(n.d.)	tail -f
Estrai colonne/campi	Select-Object / proprietà	(for /f)	cut / awk
Sostituisci testo	-replace (operatore)	(n.d.)	sed
Pulisci schermo	Clear-Host	cls	clear
Confronta due insiemi	Compare-Object	fc	diff

Si noti l'asimmetria: per molte operazioni di testo Linux ha un comando dedicato e conciso, mentre PowerShell usa un cmdlet generico (**Select-Object**, **Where-Object**) applicato a oggetti. Il codice PowerShell è spesso più verboso ma più esplicito e riutilizzabile; quello bash più compatto ma legato al formato testuale dell'output.

CAPITOLO 51

Equivalenze: processi, servizi e sistema

La gestione di processi, servizi e informazioni di sistema: qui PowerShell è particolarmente ricco, perché nasce proprio per l'amministrazione.

Tabella 51.1 - processi, servizi e amministrazione del sistema.

Compito	PowerShell	cmd (DOS)	Linux (bash)
Elenca processi	Get-Process	tasklist	ps aux
Termina processo	Stop-Process	taskkill	kill
Elenca servizi	Get-Service	sc query	systemctl
Avvia servizio	Start-Service	net start	systemctl start
Ferma servizio	Stop-Service	net stop	systemctl stop
Variabili d'ambiente	\$env: / Get-ChildItem Env:	set	env / printenv
Imposta variabile (sessione)	\$env:NOME = '...'	set NOME=...	export NOME=...
Data e ora	Get-Date	date / time	date
Info sistema	Get-ComputerInfo	systeminfo	uname -a
Pianifica attività	Register-ScheduledTask	schtasks	cron / crontab
Esegui come altro utente	Start-Process -Credential	runas	sudo / su
Spegni/riavvia	Stop-Computer / Restart-Computer	shutdown	shutdown / reboot

Equivalenze: rete

Tabella 51.2 - rete e diagnostica.

Compito	PowerShell	cmd (DOS)	Linux (bash)
Test raggiungibilità	Test-Connection	ping	ping
Test porta TCP	Test-NetConnection -Port	(n.d.)	nc / telnet
Configurazione IP	Get-NetIPConfiguration	ipconfig	ip addr
Tabella di routing	Get-NetRoute	route print	ip route
Connessioni attive	Get-NetTCPConnection	netstat	ss / netstat
Risoluzione DNS	Resolve-DnsName	nslookup	dig / host
Scarica un file	Invoke-WebRequest -OutFile	(n.d.)	wget / curl
Chiamata HTTP/API	Invoke-RestMethod	(n.d.)	curl

NOTA —

Diversi comandi di rete del vecchio cmd (**ipconfig**, **ping**, **netstat**) funzionano ancora dentro PowerShell, ma restituiscono **testo**. I cmdlet equivalenti (**Get-NetIPConfiguration**, **Test-Connection**, **Get-NetTCPConnection**) restituiscono invece **oggetti** filtrabili e ordinabili: in uno script sono molto più comodi e affidabili.

CAPITOLO 52

Confronto delle caratteristiche

Oltre ai singoli comandi, è utile confrontare le shell sulle caratteristiche strutturali. La tabella seguente sintetizza i tratti principali; le note che seguono li commentano.

Tabella 52.1 - confronto delle caratteristiche strutturali.

Caratteristica	PowerShell	cmd (DOS)	Linux (bash)
Modello della pipeline	Oggetti .NET	Testo	Testo
Tipizzazione dei dati	Sì (tipi .NET)	No	Limitata
Linguaggio di scripting	Ricco e strutturato	Molto limitato	Potente
Gestione errori	try/catch, oggetti errore	errorlevel	trap, exit code
Multiplatforma	Sì (Win/Linux/macOS)	Solo Windows	Unix/Linux/macOS
Amministrazione remota	Integrata (Remoting/SSH)	Limitata	SSH
Accesso a librerie	Intero .NET	Quasi nullo	Comandi esterni
Ecosistema pacchetti	PowerShell Gallery	Nessuno	Gestori di pacchetti
Manipolazione testo	Buona	Scarsa	Eccellente
Curva di apprendimento	Media	Bassa	Media-alta
Verbosità dei comandi	Alta (Verbo-Nome)	Bassa	Bassa
Maturità / diffusione	Alta (dal 2006)	Storica	Altissima

Punti di forza di ciascuna shell

- **PowerShell** - imbattibile per amministrare Windows e l'ecosistema Microsoft (Active Directory, server, servizi cloud), per lavorare con dati strutturati e per chi vuole un linguaggio di scripting solido e tipizzato. Oggi anche multiplatforma.
- **bash (Linux)** - lo standard de facto sui sistemi Unix/Linux, con un patrimonio sterminato di strumenti testuali, una diffusione capillare e una community immensa. Insostituibile in ambienti server Linux e DevOps.
- **cmd (DOS)** - oggi essenzialmente storico: utile per piccoli batch legacy e per compatibilità, ma superato sia da PowerShell sia, dove serve, da altre soluzioni. Resta presente in ogni Windows.

Quale scegliere

La risposta dipende dal contesto, non da una classifica assoluta:

1. **Ambiente prevalentemente Windows / Microsoft** - PowerShell è la scelta naturale: amministrazione, automazione, integrazione con i servizi Microsoft.
2. **Ambiente prevalentemente Linux** - bash (o un'altra shell Unix) è la via maestra; PowerShell resta un'opzione valida per chi gestisce flotte miste o proviene dal mondo Windows.
3. **Ambiente eterogeneo** - PowerShell 7, essendo multiplatforma, può fare da linguaggio comune; in molti team convivono comunque entrambe le shell, ciascuna dove rende di più.
4. **Manutenzione di script legacy** - si resta sullo strumento con cui sono scritti (batch cmd o script bash), migrando gradualmente ciò che conviene.

SUGGERIMENTO —

Per un tecnico dell'automazione su piattaforma Windows, investire su PowerShell è quasi sempre la scelta a maggior ritorno: copre amministrazione, integrazione con dispositivi e servizi, reportistica e notifiche - tutto in un solo strumento già presente sui sistemi, senza installare nulla.

CAPITOLO 53

Migrare verso PowerShell

Chi arriva da cmd o da bash può riusare gran parte di ciò che già sa, purché tenga a mente alcune differenze chiave. Ecco le indicazioni pratiche per una transizione senza sorprese.

Da cmd/batch a PowerShell

- I comandi DOS storici (`dir`, `copy`, `cd`, `ping`...) per lo più **funzionano ancora** dentro PowerShell, quindi si può migrare in modo graduale.
- Le variabili cambiano sintassi: da `%NOME%` di batch a `$NOME` di PowerShell; le variabili d'ambiente da `set` a `$env:NOME`.
- I confronti usano operatori testuali (`-eq`, `-gt`, `-lt`), non i simboli; gli operatori di reindirizzamento (`>`, `>>`) restano simili.
- Il salto di valore e passare dal testo agli **oggetti**: invece di analizzare l'output con `for /f`, si accede alle proprietà dei cmdlet.

Esempio 53.1 - dallo stesso compito in batch al corrispettivo PowerShell.

```
REM --- batch cmd (storico) ---
@echo off
set CARTELLA=C:\log
for %%F in (%CARTELLA%\*.log) do echo %%F

# --- equivalente PowerShell, su oggetti ---
$cartella = 'C:\log'
Get-ChildItem $cartella -Filter '*.log' | ForEach-Object { $_.FullName }
```

Da bash a PowerShell

- Concetti come pipeline, reindirizzamento e variabili esistono in entrambe: l'impianto mentale si trasferisce bene.
- Dove in bash si concatenano `grep/awk/cut/sed` su testo, in PowerShell si filtra e si proiettano **proprietà** di oggetti con `Where-Object` e `Select-Object`.
- Gli alias unix-like (`ls`, `cp`, `cat`, `rm`) aiutano all'inizio, ma hanno opzioni diverse: per gli script si passa ai cmdlet completi.
- La gestione degli errori è più ricca (`try/catch` con oggetti errore) rispetto agli exit code; vale la pena adottarla.

Esempio 53.2 - da una pipeline di testo bash a una pipeline di oggetti.

```
# --- bash ---
# ps aux | grep nginx | awk '{print $2}'

# --- PowerShell, accedendo alla proprietà Id dell'oggetto ---
Get-Process -Name 'nginx' -ErrorAction SilentlyContinue |
  Select-Object -ExpandProperty Id
```

NOTA —

La buona notizia per chi conosce più shell: i concetti si trasferiscono. Pipeline, reindirizzamento, variabili, cicli e condizioni esistono ovunque. Cambiano sintassi e, soprattutto in PowerShell, cambia la 'sostanza' che scorre nella pipeline - da testo a oggetti. Capito questo, la migrazione è in gran parte questione di vocabolario.

Fonti

- Microsoft Learn, *about_Aliases*, *about_Comparison_Operators* - learn.microsoft.com/powershell
- Microsoft Learn, *PowerShell for Bash/CMD users* (documentazione di migrazione) - learn.microsoft.com/powershell

APPENDICI

Appendici

Materiale di consultazione rapida: casi reali di automazione industriale come script completi e commentati, riferimenti rapidi ai cmdlet, una sintesi di operatori e variabili, un glossario e la bibliografia completa.

APPENDICE A

Casi reali di automazione industriale

Questa appendice raccoglie script **completi e commentati**, ispirati a esigenze concrete di un reparto di automazione (trattamento acque, robotica, PLC, SCADA). Sono pensati come punti di partenza da adattare, non come prodotti finiti: vanno provati in un ambiente di test prima dell'uso reale.

A.1 - Analisi di un log esportato da PLC

Scenario: un controllore esporta periodicamente un file di log testuale. Lo script lo analizza con espressioni regolari, estrae gli allarmi e produce un riepilogo strutturato, esportabile in CSV.

Esempio A.1 - parsing regex di un log di impianto con riepilogo.

```
<#
.SYNOPSIS  Estrae e riepiloga gli allarmi da un log testuale di impianto.
#>
param(
    [Parameter(Mandatory)][string]$LogPath,
    [string]$OutCsv = 'D:\report\allarmi.csv'
)

# Righe tipo: 2026-05-12 08:31:07 ALARM [POMPA_2] Sovracorrente
$rx = '^(<data>\d{4}-\d{2}-\d{2})\s+(<ora>\d{2}:\d{2}:\d{2})\s+' +
    '(<liv>ALARM|WARN|INFO)\s+\[(?<tag>^\)]+\]\s+(?<msg>.+)$'

$eventi = foreach ($riga in Get-Content -Path $LogPath) {
    if ($riga -match $rx) {
        [pscustomobject]@{
            Istante = [datetime]"$($Matches.data) $($Matches.ora)"
            Livello = $Matches.liv
            Tag      = $Matches.tag
            Messaggio = $Matches.msg
        }
    }
}

# Riepilogo: quanti allarmi per tag, solo livello ALARM
$eventi | Where-Object Livello -eq 'ALARM' |
    Group-Object Tag | Sort-Object Count -Descending |
    Select-Object @{N='Tag';E={$_.Name}}, Count | Format-Table -AutoSize

$eventi | Export-Csv -Path $OutCsv -NoTypeInfoation -Encoding UTF8
Write-Host "Estratti $($eventi.Count) eventi -> $OutCsv"
```

A.2 - Backup con rotazione e ritenzione

Scenario: archiviare ogni notte i file di una cartella e cancellare gli archivi più vecchi di N giorni, mantenendo lo storico sotto controllo.

Esempio A.2 - backup compresso con politica di ritenzione.

```
param(
    [string]$Origine = 'C:\impianto\dati',
    [string]$Archivio = 'D:\backup',
    [int]$GiorniRitenzione = 30
)
$stamp = Get-Date -Format 'yyyyMMdd_HH:mm:ss'
$dest = Join-Path $Archivio "dati_$stamp.zip"

try {
    # Comprime l'intera cartella in un archivio datato
    Compress-Archive -Path (Join-Path $Origine '*') -DestinationPath $dest -Force
    Write-Host "Creato $dest"

    # Ritenzione: elimina gli archivi piu vecchi della soglia
    $limite = (Get-Date).AddDays(-$GiorniRitenzione)
    Get-ChildItem $Archivio -Filter 'dati *.zip' |
        Where-Object LastWriteTime -lt $limite |
        Remove-Item -Verbose
}
catch {
    Write-Error "Backup fallito: $($_.Exception.Message)"
    exit 1
}
```

A.3 - Monitoraggio di un server SCADA con report e avviso

Scenario: controllare periodicamente lo stato di un server di supervisione - servizi critici, spazio disco, raggiungibilit  - e inviare un avviso se qualcosa non   a posto.

Esempio A.3 - health-check di un server con notifica condizionata.

```
param([string]$Server = 'SRV-SCADA-01', [int]$MinDiscoGB = 20)
$problemi = [System.Collections.Generic.List[string]]::new()

# 1) Raggiungibilit 
if (-not (Test-Connection $Server -Count 1 -Quiet)) {
    $problemi.Add("$Server non raggiungibile")
}
else {
    # 2) Servizi critici
    $servizi = Invoke-Command -ComputerName $Server -ScriptBlock {
        Get-Service 'MSSQLSERVER','W3SVC' | Select-Object Name, Status
    }
    foreach ($s in $servizi) {
        if ($s.Status -ne 'Running') { $problemi.Add("Servizio $($s.Name): $($s.Status)") }
    }
    # 3) Spazio su disco C:
    $libGB = Invoke-Command -ComputerName $Server -ScriptBlock {
        [math]::Round((Get-PSDrive C).Free / 1GB, 1)
    }
    if ($libGB -lt $MinDiscoGB) { $problemi.Add("Disco C basso: $libGB GB") }
}

# Esito + eventuale notifica via webhook
if ($problemi.Count -gt 0) {
    $testo = "PROBLEMI su $Server:`n" + ($problemi -join "`n")
    Write-Warning $testo
    $body = @{ text = $testo } | ConvertTo-Json
    Invoke-RestMethod -Uri (Get-Secret -Name 'Notifiche/WebhookOT' -AsPlainText) `
        -Method Post -Body $body -ContentType 'application/json'
}
else { Write-Host "$Server: tutto regolare" }
```

A.4 - Raccolta misure via API e archiviazione storica

Scenario: leggere periodicamente le misure da un gateway REST e accodarle a un file storico CSV, pronto per analisi successive o per l'import in un database.

Esempio A.4 - raccolta misure da API con storico CSV incrementale.

```
param(
    [string]$ApiBase = 'http://192.168.10.50/api/v1',
    [string]$Storico = 'D:\storico\misure.csv'
)
try {
    $token = Get-Secret -Name 'GatewayOT/Token' -AsPlainText
    $intest = @{ Authorization = "Bearer $token" }
    $misure = Invoke-RestMethod -Uri "$ApiBase/misure" -Headers $intest

    $righe = $misure | Select-Object `
        @{N='Istante';E={Get-Date -Format 's'}}; tag, valore, unita

    # Crea l'intestazione solo la prima volta, poi accoda
    if (-not (Test-Path $Storico)) {
        $righe | Export-Csv $Storico -NoTypeInfo -Encoding UTF8
    } else {
        $righe | Export-Csv $Storico -NoTypeInfo -Encoding UTF8 -Append
    }
    Write-Host "Accodate $($righe.Count) misure a $Storico"
}
catch { Write-Error "Raccolta fallita: $($_.Exception.Message)"; exit 1 }
```

A.5 - Creazione massiva di operatori in Active Directory

Scenario: a inizio stagione si devono creare molti account per gli operatori, leggendoli da un CSV fornito dall'ufficio personale, con log dell'esito di ciascuno.

Esempio A.5 - creazione utenti AD da CSV con conteggio esiti.

```
param([string]$Csv = 'C:\hr\operatori.csv') # colonne: Nome,Sam,OU
Import-Module ActiveDirectory
$pwd = ConvertTo-SecureString 'Tempor@neal!' -AsPlainText -Force
$ok = 0; $ko = 0

foreach ($u in Import-Csv $Csv) {
    try {
        New-ADUser -Name $u.Nome -SamAccountName $u.Sam -Path $u.OU `
            -AccountPassword $pwd -Enabled $true -ChangePasswordAtLogon $true `
            -ErrorAction Stop
        Write-Host "OK $($u.Sam)"; $ok++
    }
    catch {
        Write-Warning "KO $($u.Sam): $($_.Exception.Message)"; $ko++
    }
}
Write-Host "Completato: $ok creati, $ko falliti"
```

ATTENZIONE —

Tutti gli script di questa appendice toccano sistemi reali. Prima dell'uso in produzione: provarli su dati e ambienti di test, sostituire i percorsi e gli indirizzi di esempio, gestire i segreti con un vault (mai password in chiaro) e verificare i permessi. La prudenza qui non è burocrazia: è ingegneria.

APPENDICE B

Riferimento rapido dei cmdlet

Una selezione dei cmdlet più utili, raggruppati per attività. Non è l'elenco completo (sono migliaia), ma copre ciò che serve nella stragrande maggioranza degli script. Per ogni cmdlet, **Get-Help** fornisce dettagli ed esempi.

Esplorazione e aiuto

Tabella B.1 - esplorazione del sistema e aiuto.

Cmdlet	A cosa serve
Get-Command	Cerca i comandi disponibili (per nome, verbo, modulo)
Get-Help	Mostra la guida di un comando (-Examples, -Full, -Online)
Get-Member	Elenca proprietà e metodi di un oggetto
Update-Help	Scarica/aggiorna i file di guida
Get-Module	Moduli caricati o disponibili
Get-Verb	Elenco dei verbi approvati

File, percorsi e contenuti

Tabella B.2 - operazioni su file e percorsi.

Cmdlet	A cosa serve
Get-ChildItem	Elenca file e cartelle (alias dir, ls)
Get-Item / Set-Item	Ottiene/imposta un elemento o le sue proprietà
New-Item / Remove-Item	Crea/elimina file e cartelle
Copy-Item / Move-Item	Copia/sposta elementi
Get-Content / Set-Content	Legge/scrive il contenuto di un file
Add-Content	Accoda contenuto a un file
Test-Path	Verifica l'esistenza di un percorso
Join-Path / Split-Path	Compone/scompone percorsi in modo sicuro

Pipeline e gestione degli oggetti

Tabella B.3 - i cmdlet che lavorano sulla pipeline.

Cmdlet	A cosa serve
Where-Object	Filtra gli oggetti per condizione
Select-Object	Sceglie proprietà, prime/ultime N, valori unici
Sort-Object	Ordina
Group-Object	Raggruppa per valore di una proprietà
Measure-Object	Conta, somma, media, min/max
ForEach-Object	Esegue un blocco per ogni oggetto

Cmdlet	A cosa serve
Compare-Object	Confronta due insiemi di oggetti
Format-Table / Format-List	Formatta l'output per la lettura

Sistema, servizi e processi

Tabella B.4 - amministrazione del sistema.

Cmdlet	A cosa serve
Get-Process / Stop-Process	Elenca/termina processi
Get-Service / Start/Stop/Restart-Service	Gestisce i servizi
Get-ComputerInfo	Informazioni sul sistema
Get-WinEvent	Interroga i registri eventi
Get-CimInstance	Interroga WMI/CIM (hardware, OS, ...)
Start-Process	Avvia un programma (anche come altro utente)
Get/Restart/Stop-Computer	Gestione della macchina

Dati, conversioni e rete

Tabella B.5 - dati, conversioni e rete.

Cmdlet	A cosa serve
Import-Csv / Export-Csv	Legge/scrive file CSV come oggetti
ConvertTo-Json / ConvertFrom-Json	Converte tra oggetti e JSON
ConvertTo-Html	Genera report HTML
Select-String	Cerca testo (simile a grep)
Invoke-RestMethod	Chiama API REST (JSON -> oggetti)
Invoke-WebRequest	Richieste HTTP con risposta completa
Test-Connection / Test-NetConnection	Diagnostica di rete

APPENDICE C

Sintassi rapida: operatori e variabili

Una sintesi degli operatori e delle variabili automatiche più usati, da tenere a portata di mano. Ricordando la regola d'oro dei confronti: in PowerShell si usano operatori testuali (`-eq`, `-lt`), non i simboli.

Operatori di confronto

Tabella C.1 - operatori di confronto.

Operatore	Significato	Esempio
<code>-eq</code> / <code>-ne</code>	Uguale / diverso	<code>\$x -eq 5</code>
<code>-gt</code> / <code>-ge</code>	Maggiore / maggiore-uguale	<code>\$x -gt 0</code>
<code>-lt</code> / <code>-le</code>	Minore / minore-uguale	<code>\$x -le 100</code>
<code>-like</code> / <code>-notlike</code>	Confronto con caratteri jolly	<code>\$n -like 'PLC*'</code>
<code>-match</code> / <code>-notmatch</code>	Confronto con espressione regolare	<code>\$s -match '\d+'</code>
<code>-contains</code> / <code>-in</code>	Appartenenza a un insieme	<code>\$set -contains 'a'</code>
<code>-replace</code>	Sostituzione (regex)	<code>\$s -replace 'a','b'</code>

Operatori logici, aritmetici e di assegnazione

Tabella C.2 - operatori logici, aritmetici, assegnazione e pipeline.

Operatore	Significato	Esempio
<code>-and</code> / <code>-or</code> / <code>-not</code>	Congiunzione / disgiunzione / negazione	<code>(\$a -gt 0) -and (\$b -lt 9)</code>
<code>+</code> / <code>-</code> / <code>*</code> / <code>%</code>	Aritmetici (somma, ..., modulo)	<code>10 % 3</code>
<code>=</code> / <code>+=</code> / <code>-=</code>	Assegnazione e assegnazione composta	<code>\$i += 1</code>
<code>-f</code>	Formattazione di stringa	<code>'{0:N1}' -f 3.14159</code>
<code> </code>	Pipeline (passa oggetti)	<code>Get-Process Sort CPU</code>
<code>></code> / <code>>></code>	Reindirizza l'output a file	<code>\$x > out.txt</code>

Variabili automatiche utili

Tabella C.3 - variabili automatiche di uso frequente.

Variabile	Contenuto
<code>\$_</code> / <code>\$PSItem</code>	L'oggetto corrente nella pipeline
<code>\$?</code>	Esito dell'ultima operazione (vero/falso)
<code>\$Error</code>	Elenco degli errori recenti (<code>\$Error[0]</code> = l'ultimo)
<code>\$LASTEXITCODE</code>	Codice di uscita dell'ultimo programma esterno
<code>\$PSVersionTable</code>	Versione e dettagli di PowerShell in uso
<code>\$PROFILE</code>	Percorso del file di profilo
<code>\$args</code>	Argomenti non nominati passati a funzione/script
<code>\$null</code> / <code>\$true</code> / <code>\$false</code>	Valori nullo e booleani

Variabile	Contenuto
\$PWD / \$HOME	Cartella corrente / cartella utente

Frammenti utili da ricordare

Esempio C.1 - one-liner ricorrenti, pronti da adattare.

```
# I 5 processi che consumano piu CPU
Get-Process | Sort-Object CPU -Descending | Select-Object -First 5

# Tutti i .log modificati nelle ultime 24 ore, dimensione in MB
Get-ChildItem 'C:\log' -Filter '*.log' -Recurse |
  Where-Object LastWriteTime -gt (Get-Date).AddDays(-1) |
  Select-Object Name, @{N='MB';E={[math]::Round($_.Length/1MB,2)}}

# Eseguire un blocco su piu server e raccogliere gli esiti
'S1','S2' | ForEach-Object { Invoke-Command -ComputerName $_ -ScriptBlock { hostname } }

# Misurare quanto dura un'operazione
Measure-Command { Start-Sleep -Seconds 1 }
```


APPENDICE D

Glossario

I termini ricorrenti nella guida, in breve.

Tabella D.1 - glossario dei termini principali.

Termine	Definizione
Alias	Nome alternativo e breve di un cmdlet (es. ls per Get-ChildItem).
AMSI	Interfaccia che consente all'antivirus di ispezionare il codice prima dell'esecuzione.
Cmdlet	Comando nativo di PowerShell, con nome Verbo-Nome, che opera su oggetti.
DSC	Desired State Configuration: descrizione dichiarativa dello stato voluto di un sistema.
Esecuzione (policy)	Regola che stabilisce se e a quali condizioni gli script possono essere eseguiti.
Hashtable	Struttura a coppie chiave-valore (@{ chiave = valore }).
JEA	Just Enough Administration: accesso limitato ai soli comandi necessari.
Manifesto (.psd1)	File di metadati che descrive un modulo (versione, autore, esportazioni).
Mark-of-the-Web	Marchio che identifica i file provenienti da Internet (zona).
MBAP	Intestazione del protocollo Modbus TCP che precede la richiesta.
Modulo	Pacchetto di funzioni/cmdlet riutilizzabile (.psm1 + manifesto).
Oggetto	Dato strutturato con proprietà e metodi; ciò che scorre nella pipeline.
OPC UA	Standard di interoperabilità per lo scambio dati in ambito industriale.
Parametro	Ingresso nominato di un comando o di una funzione.
Pester	Framework per scrivere test automatici in PowerShell.
Pipeline	Catena di comandi in cui l'output di uno alimenta l'ingresso del successivo.
Provider	Componente che espone una risorsa (registro, certificati, ...) come un'unità navigabile.
PSCredential	Oggetto che custodisce utente e password in forma protetta.
PSScriptAnalyzer	Strumento di analisi statica dello stile e degli errori del codice.
Remoting	Esecuzione di comandi su computer remoti (via WinRM o SSH).
SecureString	Stringa mantenuta cifrata in memoria, per dati sensibili.
SecretManagement	Modulo per gestire i segreti tramite vault.
Verbo approvato	Verbo standard (Get, Set, New, ...) usato nei nomi dei cmdlet.
WinRM	Servizio Windows alla base del Remoting classico.
WMI/CIM	Infrastruttura per interrogare informazioni di sistema e hardware.

APPENDICE E

Fonti e bibliografia

La fonte primaria e ufficiale per PowerShell e la documentazione Microsoft Learn, costantemente aggiornata. Di seguito i riferimenti usati nella guida, raggruppati per tema.

Documentazione ufficiale

- Microsoft Learn - *PowerShell Documentation*: learn.microsoft.com/powershell
- Microsoft Learn - articoli *about_* (*about_Pipelines*, *about_Functions*, *about_Execution_Policies*, *about_Comparison_Operators*, *about_Remote*, *about_Modules*, *about_Signing*, *about_Logging*, e altri).
- Microsoft Learn - *Windows Server Documentation*: learn.microsoft.com/windows-server
- Microsoft Learn - *Desired State Configuration (DSC)*: learn.microsoft.com/powershell/dsc
- Microsoft Learn - *.NET API browser* (per le classi .NET usate, es. *TcpClient*, *SerialPort*, *SqlClient*): learn.microsoft.com/dotnet/api

Storia e origini

- Jeffrey Snover - *Monad Manifesto* (agosto 2002): il documento fondativo che delinea l'approccio a oggetti.
- Materiale storico Microsoft sull'evoluzione da Monad a Windows PowerShell (2006) e a PowerShell Core open source (2016).

Moduli e strumenti citati

- PowerShell Gallery - repository ufficiale dei moduli: powershellgallery.com
- Pester - framework di test: pester.dev
- PSScriptAnalyzer - analisi statica: github.com/PowerShell/PSScriptAnalyzer
- ImportExcel - generazione di file Excel: github.com/dfinke/ImportExcel
- Microsoft.PowerShell.SecretManagement e SecretStore - gestione dei segreti.

Standard e protocolli industriali

- Modbus Organization - specifiche pubbliche del protocollo Modbus (TCP e RTU): modbus.org
- OPC Foundation - specifiche OPC UA: opcfoundation.org
- IEC 62443 - serie di norme sulla sicurezza dei sistemi di automazione e controllo industriale.
- IEC 61131-3 - linguaggi di programmazione per controllori programmabili (riferimento per le analogie con lo scripting).

NOTA —

Per qualunque comando, la guida integrata e la fonte più rapida e sempre allineata alla versione in uso: `Get-Help <comando> -Online` apre la pagina ufficiale aggiornata. E il riflesso da coltivare: prima `Get-Help`, poi la rete.

Questa guida è stata scritta per offrire una base solida e pratica; PowerShell però continua a evolvere. Affiancare a questo testo la consultazione regolare della documentazione ufficiale e il modo migliore per restare aggiornati e sfruttare appieno le novità delle versioni più recenti.